

Baz: Designing and Implementing an Approachable Programming Language with an Expressive Type System

Jacob Horton

Bachelor of Science in Computer Science

The University of Bath

April 2025

Copyright

Attention is drawn to the fact that copyright of this dissertation rests with its author. The Intellectual Property Rights of the products produced as part of the project belong to the author unless otherwise specified below, in accordance with the University of Bath's policy on intellectual property (see https://www.bath.ac.uk/publications/university-ordinances/attachments/Ordinances_1_October_2020.pdf).

This copy of the dissertation has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with its author and that no quotation from the dissertation and no information derived from it may be published without the prior written consent of the author.

Declaration

This dissertation is submitted to the University of Bath in accordance with the requirements of the degree of BSc (hons) Computer Science in the Department of Computer Science. No portion of the work in this dissertation has been submitted in support of an application for any other degree or qualification of this or any other university or institution of learning. Except where specifically acknowledged, it is the work of the author.

Abstract

This project discusses the design and implementation of a new programming language - Baz - which combines a familiar and approachable syntax with the power and safety of an expressive type system. Baz successfully implements several major features, including a fully functional type checker, an algebraic type system supporting sum and product types, pattern matching, and safe nullable types. A multi-pass compiler, with a custom front-end and a C++ transpiler back-end, was designed and built from scratch. The project evaluates the effectiveness of Baz through several example programs, demonstrating that it enables elegant and robust solutions without the friction of a complex paradigm such as an ownership memory model. As a result, developers are able to focus on the logic of their programs, rather than battling the language itself.

Acknowledgements

Firstly, I would like to thank Professor Guy McCusker for being an amazing supervisor throughout my project. He has been incredibly supportive and encouraging, helping me work through many conceptual problems. I have thoroughly enjoyed our discussions, which have been very motivating, and helped shape this project. I am very grateful for the time and consideration he has provided me over the last year.

I am also very appreciative of my friends for generously sharing their time and insights with me. Their willingness to listen, offer feedback, and keep me motivated was invaluable. In particular, I would like to thank Sydney Wright for suggesting the name “Baz”, and Guy Johns for his patience and expertise in helping me brainstorm and overcome several challenges.

Contents

1. Introduction	1
2. Literature Review	1
2.1. An Argument for Static Types	1
2.2. Trends and Love for Types	2
2.3. Where Existing Languages Excel	4
2.4. Issues With Current Languages	5
2.5. Technical Foundations	6
3. Solution	7
3.1. Requirements Analysis and Specification	7
3.2. Design	8
3.2.1. Language Design	8
3.2.2. Compiler Design	11
3.3. Implementation	14
3.3.1. Type Environment	14
3.3.2. Enums	15
3.3.3. Optional Chaining	17
3.3.4. Arbitrary Ordering of User-Defined Types	19
3.3.5. Pattern Matching	20
3.4. System Testing	20
4. Investigation and Results	21
4.1. Example Type Errors	21
4.2. Areas for Improvement	22
4.3. Performance and Scalability	23
5. Critical Appraisal & Conclusions	24
5.1. Successes	24
5.2. Trade-Offs	25
5.3. Future Work	25
5.4. Final Remarks	27
6. Glossary	28
Bibliography	30
Appendix	35
A Example Programs	35
A.1 Counter	35
A.2 Greeter	35
A.3 Binary Search Tree	36
A.4 Expression Tree	38
A.5 Linked List	39
A.6 Turing Machine	41
B Automated Test Cases	45
B.1 Example Scanner Tests	45
B.2 Example Parser Test	48
B.3 Example Type Checker Tests	49
C Source Code	52
C.1 Expressions	52
C.2 Statements	54
C.3 Parser	58
C.4 Type Checking	71

1. Introduction

Programming languages with static type systems are becoming increasingly popular. Expressive type systems can be leveraged to massively improve the developer experience - preventing bugs, self-documenting code, conveying intent, and allowing for more confident refactoring. When a type system is expressive, such as in Rust or Haskell, it becomes a really pleasant experience to describe and model a given problem in terms of types.

Unfortunately, the existing languages with the most expressive type systems often come with an unfamiliar paradigm. Their type systems can be used to enable exciting approaches, including the ownership model in Rust, and the functional paradigm of Haskell. These can provide extra guarantees and make it easier to reason about a given algorithm, but also can discourage people from learning the language, and even make it difficult to work with.

I argue that an expressive type system can provide many benefits on its own, and therefore it is possible to have a more approachable programming language, whilst still reaping the benefits of the types and compile-time guarantees. The goal of this project was to design a language that feels familiar, is easy to learn, and remains low-friction to use, while still offering the power and benefits of an expressive type system. This language, which I call Baz, has successfully made major steps towards this goal by providing an expressive algebraic type system, safe null handling, and pattern matching, all implemented through a transpiler to C++.

2. Literature Review

2.1. An Argument for Static Types

Static types have demonstrated many key benefits when writing software, guiding the developer towards more robust and maintainable code. Aho (2014) claims that “type checking is an effective and well-established technique to catch inconsistencies in programs”, and in the same vein, Rath (2020) claims that “types define safe behaviour”. Due to this, static types can help catch a certain class of bugs earlier in the development process, and also reduce the number of bugs in finished software (Chiusano, 2016; Boyle, 2019; Yakov Fain, 2020; Mo, 2023; Bello, 2024; Oh and Oh, 2024). For example, it has been found that around 30% of bugs in Python software are type errors (Oh and Oh, 2022). Multiple papers have found that when adding types to programs (either in the same language or porting it to another language), that bugs have been discovered (Sagonas and Luna, 2008; Farrer, 2012). In addition, Milner (1978) argued that “well-typed expressions cannot ‘go wrong’”, and typed languages have been used as a framework for proofs (Sitnikovski, 2023), demonstrating their robustness. Therefore, by using static types to reason about a program, we can avoid writing these bugs in the first place, leading to more reliable code.

Another commonly stated advantage of static types is that they are self-documenting. Types encode meaning and intention, allowing them to add important information to areas of code, such as function signatures. For example, types can define exactly what a function expects as input and what it will return as output. Sometimes, as in the case of Rust and Haskell, these types will also encode information about the kinds of errors that can occur. This extra information for the developer can help to reduce the development time. In the case of using an API you need to know what data the API expects and what it will return. Endrikat et al. (2014) found that when using a new API, if it was written using static types, the speed of development increased when compared to dynamic types. The type information can also be used to auto-generate documentation/specifications (*How to write documentation - The rustdoc book*, 2024; OpenAPI Tools, 2024).

As a result of types being self-documenting, they can also allow code to be more maintainable. If developers are able to understand the code better, it will be easier to expand on it and spot existing

bugs. With the use of a static type system, it becomes a lot easier and faster to fix type errors. In scenarios where type errors need to be fixed, Hanenberg et al. (2013) found that development time, the number of test runs, the number of files opened, and the number of file switches were all reduced when working with a statically typed language, compared to a dynamically typed equivalent. This demonstrates that types provide the developers with more information in the areas that they need it, reducing the need to look elsewhere or test-run to check functionality.

In addition, static types help to “[track] information the programmer would otherwise have to track mentally in some fashion” (Chiusano, 2016), which means they can stay focused on the task, and avoid having to context-switch to remember some information. This is backed up by Hanenberg et al. (2013), who found that in certain tasks, static types reduced the number of file switches. This is likely due to the support IDEs can provide when types are available - showing the exact data format each function takes, even if it’s in another file.

While there are a lot of arguments for statically typed languages, it’s worth mentioning that they are not suitable for every situation. For example, dynamically typed languages sometimes have faster development times, especially when writing smaller programs (Stuchlik and Hanenberg, 2011; Mayer et al., 2012). Hanenberg et al. (2013) also found there was no statistical speed improvement in fixing semantic errors between static and dynamic languages. Despite this, I aim to argue that when code is needed to be reliable and maintainable, statically typed languages should be preferred.

2.2. Trends and Love for Types

Types are highly loved in the developer community. There are countless blog posts on the beauty of type systems in different languages (Maguire, 2015; Yakov Fain, 2020; Moskala, 2023; Jintin, 2024). Rust is a clear example of this (Albrizio, 2023), demonstrating strong community support - so much so that there are articles that explore whether the enthusiasm for the language is justified (Ortiz, 2020).

The Stack Overflow surveys (Stack Overflow, 2020; 2024) provide a lot of useful insight into the popularity of languages. An interesting metric is “desired” vs “admired” languages. A “desired” language is one that people do not currently use but would like to, and an “admired” one is a language that someone uses and wants to continue using. In the 2024 survey, JavaScript and Python were 1st and 2nd, respectively, in terms of desired languages. However, in terms of admired languages, Rust was top, followed by Elixir, Zig, and TypeScript. 3 of these 4 languages are statically typed, which indicates that people enjoy using languages due to their type systems, despite there not being as many people wanting to learn those languages. This could be due to a steep learning curve or a barrier to entry.

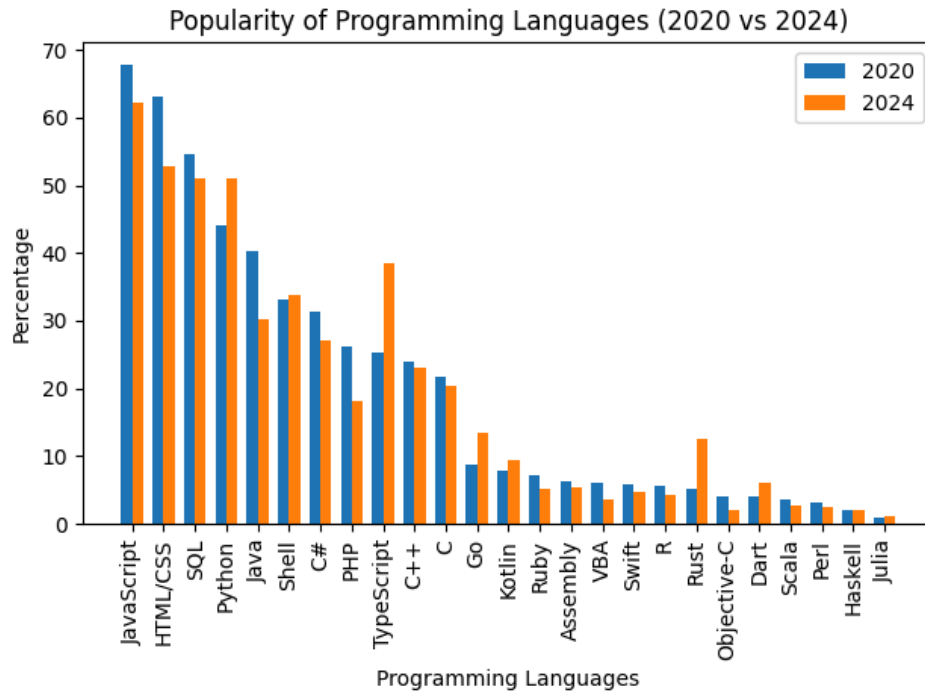


Figure 1: Stack Overflow Survey - Popularity of Programming Languages Between 2020 and 2024

Figure 1 compares the most commonly used languages between 2020 and 2024, as recorded by the Stack Overflow survey (Stack Overflow, 2020; 2024). This graph shows that many statically typed languages are on the rise and some dynamically typed languages are declining. Notably, JavaScript and PHP are both dynamically typed languages and have both reduced in popularity. There has been a huge increase in TypeScript and Rust and also increases for Go, Kotlin and Dart, which are all statically typed.

The increase in TypeScript alongside the decrease in JavaScript suggests that people are switching from JavaScript to TypeScript in order to make their code type-safe. This shift to type-safe code coincides with the JetBrains 2020 Python Developers Survey (JetBrains, 2020), which found that the most desired Python feature was static typing and strict type hinting. Both of these suggest that developers are trending towards languages that have static type checks.

Other languages are also improving their static type support. For example, since PHP 7.4, class properties support type declarations (The PHP Documentation Group, 2019), and their use has been encouraged in order to create cleaner code (Pitt, 2016). In addition, even though already statically typed, Java has become more expressive with its types over the years. Java 8 introduced an “Optional” class (Oracle, n.d.), which provides clarity on which variables might not have a value.

Several companies have mentioned a shift from PHP to Golang (Go), using Go for new code that otherwise might have been PHP (Elsinga, 2020; Simsek, 2021; Pistunovich, Boyle and Shepherd, 2024). This matches with what we see in Figure 1 - a decrease in PHP and an increase in Go. This could be due to the static typing that Go provides, but also Go provides many other benefits over PHP such as runtime speed and concurrency support.

This graph does not paint a perfect picture for static typing, however. Python has increased in popularity, and Java has decreased. Python’s increase could be explained by its use in Machine Learning (ML) (Raschka, Patterson and Nolet, 2020), and the recent boom in ML research. However, as mentioned before, Python developers are requesting static type checking, which may suggest

people are not using it specifically because of its dynamic typing. Java's decline could be explained by newer languages with more modern features, such as Kotlin, replacing it.

Furthermore, the White House recently released a paper which explained the reasons for needing safety at the language level (The White House, 2024). Some languages can prevent a programmer from making certain mistakes. In particular, they discuss the importance of memory safety for security and suggest Rust as a potential language to fit their requirements. I believe their argument for safety at a language level can also be applied to static types, as they can eliminate the possibility of a runtime type error.

2.3. Where Existing Languages Excel

Rust, Haskell and OCaml all support algebraic types. This means product types, such as structs with properties, as well as sum types, which are often implemented as enums with payloads, i.e. tagged unions. Sum types significantly increase the flexibility of the type system, by essentially allowing a variable to be one of several types, whilst remaining type-safe. This is especially useful when handling JSON data, which does not necessarily have a defined structure.

In addition, these languages also support pattern matching, which makes the process of handling algebraic types much more pleasant. A C++ proposal, written by the creator of C++ (Bjarne Stroustrup) and others, stated that “Pattern matching is a widely adopted mechanism across many programming languages to interact with algebraic data types that can help greatly simplify C++” (Murzin et al., 2019). I believe that this does not only apply to C++ but many languages with algebraic data types. Without pattern matching, the developer would need to define multiple `switch/case` or `if/else` statements, and then have to extract the required data inside each one. However, with the use of pattern matching, you can match on the data types and property values, even when nested, as well as “destructure” other properties all at the same time.

Go, OCaml, Rust, F#, and Swift all represent errors as values. Through the type system, they are explicit in where errors can occur, and how they “bubble” up functions, in contrast to exceptions, which I explain in more detail in Section 2.4. The downside of this is that there may be more boilerplate code for handling and propagating the errors. Errors as values function best when the compiler enforces that those values are checked and handled appropriately. In doing so, the programmer is forced to do something in response to an error, reducing the likelihood that they ignore the error case. Many languages with errors as values also support some form of “panic” or exceptions, but these are recommended to only be used when the error is not recoverable (Bolognese, 2012).

Go implements errors as types by using multiple return values, and ensuring all returned variables are used/checked. Rust implements it by wrapping return values in a `Result` type, which has to be unwrapped before using the value inside. Rust's approach arguably offers a more elegant solution to error handling and avoids the possibility of discarding errors silently, which is possible in Go using `_ := err`.

Interfaces/traits decouple specification from implementation, which can lead to more maintainable code, and there is a consensus amongst developers that they are a good practice (Steimann and Mayer, 2005). Golang gets a lot of praise for its interfaces (Dudeja, 2017), which work through implicit implementation - any struct with the required method signatures will implement that interface. This can further help with decoupling. On the other hand, many other languages, such as Java, Rust, Swift, and Kotlin, all require you to specify which interface you are implementing. This is useful because if it does not meet the interface requirements, the compiler can tell you before you even use the object. Therefore, this approach makes it easier to maintain the code when changes are made to interfaces.

The Stack Overflow Surveys (Stack Overflow, 2020; 2024) show that Rust is consistently the most admired language, and is growing quickly in popularity. It has used OCaml as a massive source of inspiration, as well as Haskell, and Swift (*Influences - The Rust Reference*, 2024), pulling some of the best features from each. Its strict memory model and type system encourage and guide developers into writing good code with its correctness and explicitness (*How has learning and working in Rust influenced how you think about writing software?*, 2020). Rust is well suited to low-level programming, using its guarantees to avoid the typical pain point with manual memory management, which can cause up to 70% of all bugs (The White House, 2024). The compile-time guarantees of Rust are likely a large factor in its popularity. However, these same benefits can make it really hard to write code in Rust - an issue I discuss in Section 2.4.

Null values can cause a lot of issues, which I also discuss in more detail in Section 2.4. Optional types are a nice alternative to this. They explicitly encode when values can possibly be missing (i.e. null), and by not using one, the variable is guaranteed to have a value. This can allow for more peace of mind as a developer, as you will know when you have to handle the edge case of no data. The compiler can also enforce that you handle the case of `null` in these optional values before you use the data inside. Rust, Haskell, and OCaml all support this elegantly through sum types and pattern matching, without requiring extra syntax. Kotlin also does this well with its “nullable types” (Jintin, 2024), which have several useful methods for handling null values.

Go is a fairly new language that has been very successful recently. This could be attributed to many factors, including its speed, easy concurrency, and simplicity. The philosophy of Golang, which has enabled its simplicity, has a few key aspects (Dudeja, 2017):

- Only adding new features when they are really needed - avoid having multiple ways to do the same thing
- Keep the language as simple as possible, so that it is easy to learn
- Keep the language readable - this goes hand in hand with its simplicity

I aim to keep a similar philosophy in mind, in order to create a language that is robust and readable, whilst avoiding a steep learning curve.

Finally, TypeScript (TS) was created as a superset of JavaScript (JS), including type annotations and a static type checker. It transpiles to JS to leverage the existing JS ecosystem while providing a level of safety and self-documentation, as discussed in Section 2.1. The type system is very flexible and has even shown to be Turing-complete in itself (Dieterichs, 2017). Many people argue for the advantages of TypeScript (Yakov Fain, 2020), backed up by its popularity growth in recent years (Stack Overflow, 2020; 2024). I think its flexibility is where it shines - it is not intrusive, allowing faster development times. However, this also leads to it being easy to opt out of the type system, thus compromising the benefits of the type system.

2.4. Issues With Current Languages

As mentioned in Section 2.2, there appears to be a gap between people wanting to use languages with type systems and admiring them once they have learnt them. The “strangeness budget” is a term coined by Klabnik (2015) to mean the amount of new features that a language can have before a developer decides it would cost too much to learn the new language, therefore avoiding it. I believe these languages suffer from exceeding their “strangeness budget”, which would explain their limited adoption. For example, with Rust, it is easy to find people explaining the steep learning curve of Rust (Vaner, 2018; Tietz-Sokolsaya, 2023) and struggling with the borrow checker and/or lifetimes (Shui, n.d.; Renoth, 2021). Fiala et al. (2023) state that “many familiar implementation patterns must be adapted substantially” for Rust. They go on to state that “navigating the intricacies of the Rust type system can be difficult for programmers” and “the type system of Rust, which is so effective at

preventing bugs statically, is also what makes Rust code hard to write for developers who are new to the language”. The Rust survey (The Rust Survey Team, 2020) showed that lifetimes are the trickiest topic in Rust, with over 60% of Rust users finding them “tricky” or “very difficult”. This was followed by ownership (the borrow checker) at over 45%. People often use the term “fighting the borrow checker” when describing their difficulties with ownership (*Newbie learning how to deal with the borrow checker - help*, 2020; Renoth, 2021; Shamrell-Harrington, 2022). This difficulty is off-putting to many and therefore could be hindering Rust’s growth.

Similarly with Haskell, it “may be a difficult adjustment” (Hudak and Fasel, 1992) for developers coming from an imperative language, due to its functional paradigm. It is also built on “purity”, meaning mutability is not allowed, and it is generally more abstract than imperative code, which makes it daunting to learn (Riston, 2017).

Null references have been used in several languages, including Java, but the creator of them later called them a “billion-dollar mistake” (Hoare, 2009), stating that “[nulls have] led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years.” Despite all its good qualities, Go has a nil type, which can cause the same difficulties as null. However, Go tries to improve on this using “zero values” - a sensible value for each type to initialise to. For example, an integer would be `0`, a string would be `""`. This design can be unexpected, leading to difficulties, such as when parsing JSON - null values would be “unmarshalled” as the zero value for that type. In order to avoid these difficulties, some modern languages have used “Optional” or “Maybe” types, which encode in the type that the value could be null/missing. This means the compiler can enforce that the null case is handled. As mentioned before, Java 8 introduced the “Optional” class in an attempt to remedy this.

Exceptions are useful in reducing bugs, however, they are often used badly or incorrectly, leading to many problems. Pádua and Shang (2017) discuss the anti-patterns that cause these problems. In addition, they are commonly criticised, as outlined by Wilson (2010). They break the locality of code - allowing errors to “bubble up” through functions, and potentially cause an error to occur in a place the programmer did not expect. It is also sometimes the case, as with C# (Linington, 2024), and JavaScript, that exceptions do not have to be specified in the function signature, and therefore can be missed. Kechagia and Spinellis (2014) found that 19% of crashes when using the Android API could have been avoided if exceptions had been documented. By using errors as values instead, the programmer would be forced to handle the error, or explicitly ignore it, leading to a lower chance of unexpected crashes.

2.5. Technical Foundations

In developing this language, I utilised several established resources covering several key areas:

- Language design (Pierce, 2002; Scott, 2009) - informing decisions around syntax, semantics and usability
- Compiler design (Cooper and Torczon, 2012; Aho, 2014; Nystrom, 2021) - guiding the architecture and implementation
- Static type system implementation (Pierce, 2002; Rathi, 2020) - ensuring a sound approach to the design and implementation of the type system

These sources provide both theoretical and practical insight and guided the technical development of this project.

3. Solution

3.1. Requirements Analysis and Specification

With all this in mind, I planned to develop a language with the niceties of a rich static type system, but without it being intrusive and exceeding the “strangeness budget” (Klabnik, 2015). I aimed to follow in the footsteps of Go, keeping the language simple, without much syntactic sugar, and similar to C. I also aim to build on the successes of Rust, Haskell, Swift and Kotlin with their amazing type support. I believe the key features that make these languages’ type systems so good are:

- Algebraic types
- Pattern matching
- Interfaces/traits

To avoid the “billion-dollar mistake” (Hoare, 2009), I aimed to leverage the type system to create an optional type, which could allow a value to be null. Any type that is not optional must have a value that is not `null`. I could therefore enforce that the optional type must be checked before it can be converted into its non-optional counterpart and used as a non-optional type. I also planned to represent errors as values, as done in Rust, Go, Zig, and Swift.

Property	Explanation	Purpose
C or Java-like syntax	Keep most syntax similar to C and its derivatives	Keep the syntax familiar to most developers to reduce the learning curve (stick within the “strangeness budget” (Klabnik, 2015))
Product data types	Structures, similar to structs in C. Also support methods	Encapsulation of data and functionality
Sum data types	Enums where variants can contain data, similar to enums in Rust, or tagged unions in other languages. Also support methods	Allows flexibility to represent more complex data, whilst remaining type-safe
Pattern matching	Depending on the data provided, and its contained values, execute different code branches	Type-safe, and elegant way to filter data types with specific values
Traits (interfaces) and generic types	Support for polymorphism	Decouple code and reduce duplication without creating complex inheritance trees
Type checker	Guarantee that types are correct at compile time	Reduce bugs, increase correctness
Automatic memory management	Allocate and free memory (where needed) automatically	Avoids the programmer having to understand a memory model
CLI tool to compile	CLI interface that takes a file name as input, and outputs a binary file that can be executed	Easy execution of code, and ability to run on a system without needing to install anything
Safety guarantees	Memory safety - ensure there are no accesses of unallocated memory, and no double freeing Null safety - ensure any value that	Prevent a class of bugs that are common due to lack of safety in these areas

Property	Explanation	Purpose
	can be null has been checked before being used	
Useful compile time error messages	Provide the programmer with error messages that can help them quickly identify what is wrong	Make it quicker and easier for the developer to fix bugs

3.2. Design

3.2.1. Language Design

I aimed to keep the syntax familiar, so I took inspiration from popular languages such as C and Java. I also took inspiration from more modern languages such as Rust and Zig. To get an idea for the syntax, I started by imagining what the language would look like with an example program, which can be seen in Listing 1. This could then be used to assist in reasoning about the syntax and semantics of the language. The final syntax of Baz ended up very similar to the prototype program - examples of valid programs in the final version of Baz can be seen in Appendix A.

I did not want the programmer to have to consider memory management, so I avoided any syntax that required knowledge of memory allocation/freeing, pointers, references, etc. This meant that to automatically free memory, Baz had to be either reference counted or garbage collected. As reference counting does not handle all cases (e.g. circular references), I decided that garbage collection would be the best approach. As seen in Listing 1, there is no syntax to indicate where memory is allocated or freed, despite there being a self-referential data structure that requires heap allocation. However, implementing a garbage collector is a large task and was considered a low priority, as the focus of this project was not on memory management. For now, memory is allocated as needed but never freed. While this is very impractical for large or long-running programs, it is sufficient for small proof-of-concept programs.

To encourage high cohesion, and thus cleaner code (Yourdon and Constantine, 1979), I decided that algebraic data types (structs and enums) should be able to have methods on them. This allows for related code to be grouped together, thus making it easier to follow. This can be seen in the `sum` method in Listing 1.

For safe handling of null values, Baz supports optional types. In Rust, these are expressed as an enum called `Option`, which has two variants - `Some(data)` meaning non-null, and `None` meaning null. The type definition for an optional integer would be `Option<i32>`, and to create an optional integer, you would write `let optional: Option<i32> = Some(5)`. Haskell is also very similar. I believe this approach is too verbose for a construct that is used a lot. Instead of this, I opted to use dedicated syntax - indicating optional types by a `?` after the type definition, e.g. `int?`. Baz also allows implicit conversion from a non-optional type to its optional type to avoid having to wrap it, as you would in Rust. For example, to create an optional integer in Baz, you would write `let optional: int? = 5`.

Optional types are distinct from their non-optional counterparts, allowing safe use of null values. This means they cannot be used where a non-optional type is expected. In order to use an optional value as a non-optional value, it must either be extracted through a `match` statement, or used with a fallback value. The fallback syntax is inspired by the null-coalescing operators in several languages, including C#, JavaScript, and PHP. For example, in Listing 1, the `?? 0` syntax is used to indicate that if the value is `null`, fall back to a value of `0`. This allows a small amount of code to handle null values, whilst remaining null-safe.

```

// Struct to hold the data for a tree node
struct TreeData {
    // Left and right are optional values (they can be `null`)
    left: TreeNode?;
    right: TreeNode?;
}

enum Tree {
    // Either a leaf with a value, or a node with a left and right Tree
    Leaf(int);
    Node(TreeData);

    fn sum(): int {
        match (this) {
            Tree.Leaf(value): { return value; },
            Tree.Node(tree_node): {
                // Sum the each side if it exists, or fall back to 0 if not
                return tree_node.left?.sum() ?? 0 + tree_node.right?.sum() ?? 0;
            }
        }
    }
}

fn main(): void {
    // Create a tree with two leaves
    let tree: Tree = Tree.Node(
        TreeData {
            left: Tree.Leaf(1),
            right: Tree.Node(
                TreeData {
                    left: Tree.Leaf(2),
                    right: null,
                }
            ),
        }
    );

    println(tree.sum());
}

```

Listing 1: Prototype design of syntax. This program demonstrates the use of enums to create a tree structure where only the leaves have values. The methods `sum` and `increment_leaves` walk through the tree to perform their operations.

Another area of consideration for the design was whether variables should be passed by value or by reference. To keep the language simple, I wanted to be consistent between variable assignment, function parameter binding, and variable binding in pattern matching. As pattern matching is the only way to extract the data from an enum, I initially planned to allow modification of the bound variable inside a pattern match branch. This would make the following possible:

```
match (foo) {
  Foo::Variant(value): {
    value = value + 1;
  }
}
```

However, the problem with this is that if I made it consistent with function parameter binding, then any reassignment of a parameter would be reassigned outside of the function. For example:

```
fn modify(foo: int): void {
  foo = 5;
}

let bar: int = 6;
modify(bar);
// `bar` is now 5
```

I believe this would be unexpected behaviour for most developers, especially those who use C, Java, or Python. C++ allows passing by reference, which would allow code that functions like `modify` above, but it has explicit syntax to make it clear when a value is passed by reference vs by value. Using explicit pointers or references would solve this issue, but to stay consistent with the rest of Baz, I decided to keep the syntax simple, thus reducing the amount the programmer would have to know to use Baz.

Instead, I decided to follow the approach of Java and Python, where reassignment will not update the original value, but modifying a property on an object will update the original object. Figure 2 shows how this is achieved - `foo` is a variable inside the `main` function. It references an object in memory that may have several properties on it. When calling the function `bar`, it passes `foo` as the argument. The reference `foo` is not passed itself, but instead a copy of the reference is passed. Therefore `foo_in_fn` is a distinct reference that points to the same object, which allows for reassignment of `foo_in_fn` without affecting `foo`. Any properties modified on `foo_in_fn` modifies the object that `foo` also references, meaning `foo` it will also be updated. A benefit of this is that copying a reference is really cheap, making the resulting program faster. By staying similar to popular languages in this aspect, I reduce the amount of the “strangeness budget” (Klabnik, 2015) that is being used up.

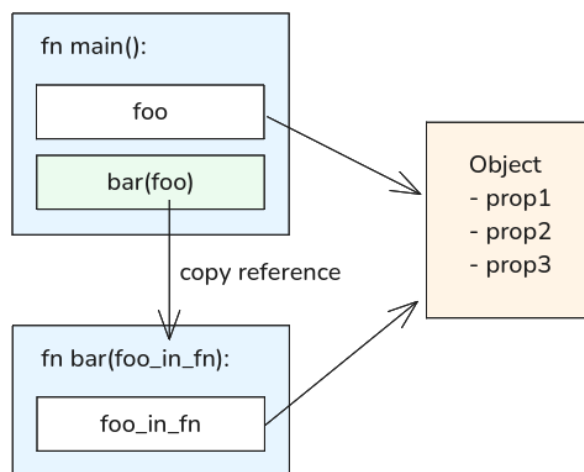


Figure 2: Passing an object reference by value

The problem with this method is that, if staying consistent, a bound variable in a pattern match cannot be modified. Instead, to update the value inside an enum variant, you will need to reassign the original variable:

```
match (foo) {  
  Foo::Variant(value): {  
    foo = Foo::Variant(value + 1);  
  },  
}
```

Unfortunately, it is often useful to match on `this` inside an enum method. To be able to use the above code on `this`, as shown in Listing 2, Baz needs to allow the reassignment of `this` to affect all other references. Therefore, I have made an exception to pass `this` by reference, while keeping all the other function parameters as pass object reference by value. While not as elegant as I had hoped, this approach allows me to stay closest to my initial goal - creating a familiar language without the difficulty of understanding pointers and references.

Furthermore, I decided to make pattern matching exhaustive, causing a compile time error if any cases are missing. This allows more bugs to be caught earlier on - at compile-time, rather than run-time - especially when maintaining code. For example, if a new variant is added to an enum, it is often the case that the programmer needs to update all match statements for that enum to cover the new case. If the compiler enforces that all cases are covered, this ensures that the new variant is handled properly in every match statement.

3.2.2. Compiler Design

I decided to implement Baz using a multi-pass compiler. In a multi-pass compiler, there are distinct phases that each take the output from the previous phase as their input. This approach has several

```
enum OptionalNumber {  
  Some(int);  
  None;  
  
  // Increments the value of `Some`, otherwise does nothing  
  fn increment() {  
    match (this) {  
      OptionalNumber::Some(value): {  
        // Reassign `this` so we can update the value  
        // Note: `this` is a reference, so reassigning will also reassign all  
other references  
        this = OptionalNumber::Some(value + 1);  
      },  
      OptionalNumber::None: {},  
    }  
  }  
}  
  
fn main(): void {  
  let num: OptionalNumber = OptionalNumber::Some(5);  
  num.increment();  
  // `num` is now `OptionalNumber::Some(6)`  
}
```

Listing 2: Reassigning `this` within a method

benefits, including easier analysis of the code, and easier implementation of more complex language features, when compared to a single-pass compiler (Cooper and Torczon, 2012).

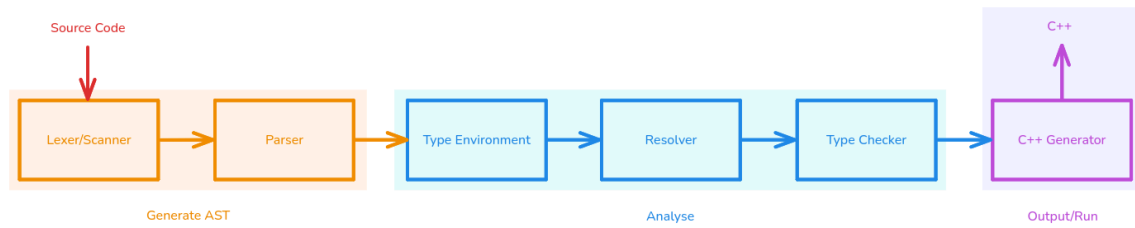


Figure 3: Overview of compiler architecture

As seen in Figure 3, the general flow involves:

1. Converting the source code into a more useful, abstract form - an Abstract Syntax Tree (AST)
2. Performing analysis on the AST - this includes type-checking
3. Running the code directly, or producing an output that a computer can run

The first two points both fall under the umbrella of the “front end” of the compiler, and the last under the “back end”. The front end is concerned with understanding the code, and the back end with generating something a computer can run. I found the book *Crafting Interpreters* (Nystrom, 2021) to be an especially useful resource in developing the front end of my implementation.

To convert the source code into a more useful representation, the compiler performs two phases - scanning and parsing. Scanning takes the individual characters from the source code and turns them into “tokens”, which makes parsing easier. Each token is one or more characters that could be a keyword, symbol, identifier etc. The token stores information such as its type, the lexeme (original text value), and the line number (used for error reporting). Then the parser can take the tokens and generate an Abstract Syntax Tree (AST), which describes the structure of the program and, in my case, also the order of operations. For example, Figure 4 shows the AST generated for the expression `5 + 4 * 6`. It describes that the operation `4 * 6` should be performed first, and the result of that used in the addition with `5`. Each node here shows the lexeme, but in reality, the whole token data is stored, often with some extra information such as the types.

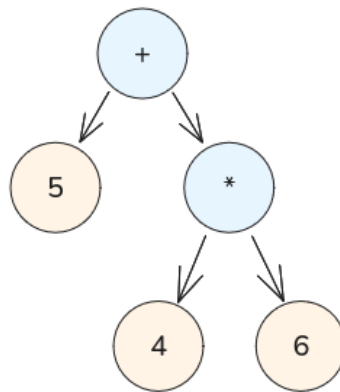


Figure 4: Abstract Syntax Tree of `5 + 4 * 6`

With an AST, the compiler can more easily analyse the program. In my implementation, this also involved several phases. The first pass looks for any user-defined types, which includes structs and enums, and gathers these into a “type environment” to be used for type-checking later. The type environment contains every possible type and information about the type, including the primitive types (integers, floats, booleans, etc.). Following this is a resolving phase, which looks for all variable uses and connects them to their declarations. With this information, the type checker can know the

type of every variable usage, which is essential to perform type-checking of the whole program. The final step in the analysis is the type-checking pass, which verifies the correctness of types in each statement and expression. This uses the information from the type environment as well as the resolver.

To perform type-checking, I used a systematic approach, utilising the structure of the AST. This follows the formal definition of typing judgements, as seen in Pierce (2002). Each expression in Baz has an associated type, which represents the type of the value produced by evaluating the expression. Compound expressions are built from sub-expressions, each of which has its own type, forming the premises for their typing rules.

The AST stores this hierarchical structure and type information in a tree format, which facilitates traversal during type-checking. The type checker uses a bottom-up approach, beginning with the leaves which have a fixed type, such as literals and variable uses. These correspond to axioms, which are typing rules without premises. For literals, the type is straightforward to determine - it is known at the scanning stage from the format of the literal (e.g. floats have a decimal point, strings are surrounded in quotes, etc.). For variables, it uses the information from the resolving step to directly know the type.

The next step of the type checker examines the immediate parent nodes, checking the compound expressions. Each kind of expression has an associated typing rule, which specifies the required types of its sub-expressions and determines the resulting type. For example, Equation 1 shows the typing rule for the addition of integers in Baz. It states that both the left (t_1) and right (t_2) sides must be integers, and the resulting expression evaluates to an integer.

$$\frac{\Gamma \vdash t_1 : \text{int} \quad \Gamma \vdash t_2 : \text{int}}{\Gamma \vdash t_1 + t_2 : \text{int}} \quad (\text{T-ADD}) \quad (1)$$

The type checker continues this process until reaching the top-level expressions. If at any point, a type rule does not hold, the type-checking phase fails, reporting a type error. Baz also supports implicit coercions, allowing the checker to validate that one type can be implicitly converted into another. For example, this can support the implicit coercion of non-optional types to optional types. Once the type checker works out the type for a given expression, it annotates the AST node for that expression, which enables later phases to easily access the type information.

After annotating the type of each expression, the type checker then looks at statements to ensure their types are also valid. This includes variable declarations, reassignment, if/for/while statements, and pattern matching. Each of these will ensure the types make sense and will error if they do not. The key difference between statements and expressions is that expressions have a resulting value, whereas statements do not. This means that statements do not have a type, and no annotations are made on the AST for them.

Using this approach, the type checker is able to ensure that:

- A variable remains the same type after reassignment
- Function calls have the correct number of arguments, all of the correct type
- All values returned from a function have the correct type
- If not void, functions will always return a value
- Structs are initialised with values for all of their properties
- When accessing a property on a struct, it actually exists
- Pattern matching is exhaustive - the patterns cover every possible case

For the back end of my implementation, I decided to output to C++ code, thus making it a transpiler, rather than a compiler. This method was chosen for several reasons:

1. Simplicity - it was comparatively easy to implement compared to machine code - the focus of my project was on the type system, and therefore it would not be beneficial to learn about machine code or LLVM
2. Performance - C++ compilers have lots of optimisations that I could leverage without extra work
3. Compatibility - C++ code can be compiled for almost any platform
4. Binary file output - the final output is a binary file, which can be run directly without the need for installing any software, such as a virtual machine

Although a tree-walk interpreter may have been easier to implement, it would have the disadvantages of worse performance and compatibility, and the need to install software to run a Baz program.

3.3. Implementation

As mentioned in Section 3.2, I began with an example program and used that to think through the syntax and semantics of the language before it was implemented. The implementation was then performed incrementally for each feature.

When starting the implementation, I knew most of the language syntax, so I was able to develop most of the scanner and parser. I chose to implement these from scratch, instead of using tools such as YACC and Lex, for two reasons - I wanted to learn how a parser is made, and I would have more flexibility and control over the details and data format. I implemented the parser using a recursive descent algorithm. From this, I was able to generate an AST for most programs in Baz. Afterwards, I built up enough of the following phases (e.g. the type checker and C++ generator) to get an initial end-to-end test working for the core language syntax e.g. basic statements and expressions, control statements, and functions. I did this so I could check that the pipeline I designed worked and fit my needs.

From this point, I worked on a language feature at a time (e.g. structs, enums, pattern matching, optional types), getting each working through all phases of the transpiler. This made it easy to validate the transpiler incrementally. Moreover, it created a tighter feedback loop, facilitating easier revisions of both the language design and compiler architecture. It became quicker to spot when a feature's semantics or implementation was awkward or frustrating to work with.

In terms of my approach to development, I used Git for version control, publishing the code to [GitHub](#). The transpiler was written in C++ on Linux, without the use of any external libraries. CMake was used to facilitate the compilation of the transpiler, therefore allowing it to be compiled on Linux, Mac, and Windows. Additionally, as Baz transpiles to C++, it should be possible to run Baz code on any of the same three platforms.

3.3.1. Type Environment

When developing the struct feature, I had to rework the way I handled types. Initially, the resolver, on encountering a type, would immediately attempt to look up its definition in the accessible scopes. However, as the resolver only performs one pass, it enforced that definitions must come lexically before the type usage, even if the types were in scope. As a result, it was not possible to arbitrarily order structs, and it made it impossible to define mutually recursive types, as shown in Listing 3. It can be seen that whether struct `A` or `B` is defined first, it has a reference to the other type, which will not have been encountered by the resolver yet.

```

struct A {
    // Resolver will look up type definition of type `B`
    b: B;
}

struct B {
    // Resolver will look up type definition of type `A`
    a: A;
}

```

Listing 3: Mutually recursive types `A` and `B` - each refers to the other

Fortunately, the entire type definition is not required in the resolving phase, and therefore the resolution of the type can be deferred. Instead, the resolver annotates each variable and property with a symbol - specifically the lexeme (name) of the type - to find the type information later. At the same time, all user-defined types are collected into a type environment - a table to look up the full type information from the symbol. This decouples the use of a type from the place it is declared, solving the ordering problem. Later stages, such as the type checker, can then look up type symbols in the type environment to find the full information needed for type validation. Figure 5 demonstrates an example AST and associated type environment. It can be seen that the variable expression `foo` is annotated with the type symbol `Foo`. This is not a reference to `Foo` in the type environment directly, but a symbol which matches the key in the type environment.

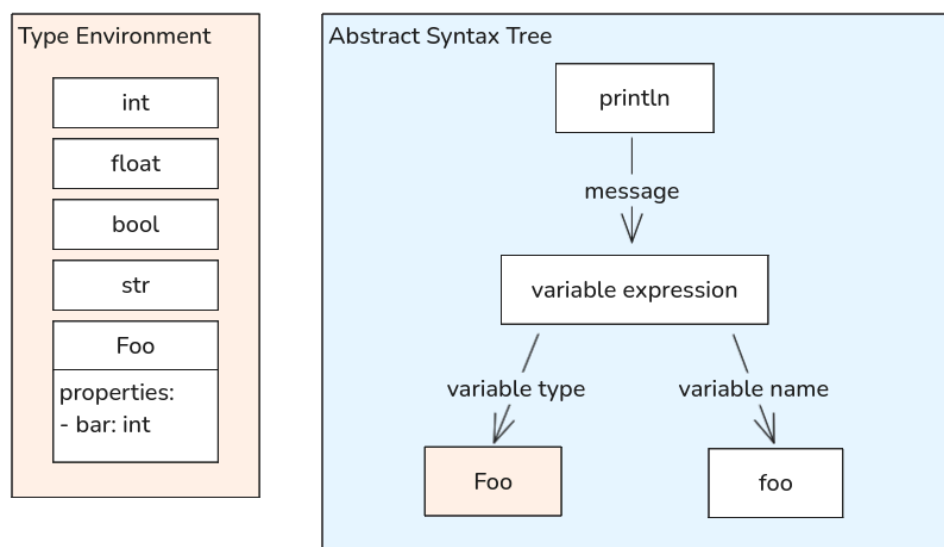


Figure 5: Abstract syntax tree and type environment for the snippet `println(foo)` where `foo` has type `Foo`

3.3.2. Enums

The implementation details of enums also had some difficulties. The first hurdle to overcome was the syntax - originally, I had planned to use the syntax `EnumName.EnumVariantName` in order to keep the syntax simple. However, this is the same syntax as getting a property on a struct (`object.property`). To support this, when any phase of the transpiler encountered the `.` syntax, it would need to first check whether it was operating on an object or the name of an enum. This is true for the resolver, type checker, and C++ generator. Therefore, there would have to be two branches of code to handle each scenario in each of these areas. I decided that this would be unnecessary implementation complexity, so to keep the code neater and reduce the chance of subtle bugs, I

decided to switch to a different syntax for enums: `EnumName::EnumVariantName`. This allowed the transpiler to know at the parser phase that this was an enum variant and not a property lookup.

The second hurdle of enums was in the C++ generation. Unlike structs, C++ does not have a direct equivalent of enums that can hold data. Instead, I opted to use `std::variant` with a struct for each variant, where the struct could contain the data or “payload”. For example, the following code in Baz:

```
enum Message {  
    Text(str);  
    Number(int);  
    Empty;  
}
```

will generate the following C++ code:

```
namespace Baz {  
    struct Message_Text { std::string value; };  
    struct Message_Number { int value; };  
    struct Message_Empty {};  
}  
  
using Message = std::variant<Baz::Message_Text, Baz::Message_Number,  
    Baz::Message_Empty>;
```

Note that these structs are generated by the transpiler, and therefore are put in the `Baz` namespace to avoid name collisions. They are also namespaced by the enum name (`Message` in this case), so that the programmer can have multiple enums with the same variant name.

Pattern matching can then be done using `std::holds_alternative`, and the data can be extracted using `std::get`. For example, the following code in Baz:

```
match (message) {  
    Message::Text(text): {  
        print("Got text: ");  
        println(text);  
    },  
    Message::Number(num): {  
        print("Got number: ");  
        println(num);  
    },  
    Message::Empty: {  
        println("Empty");  
    },  
}
```

will produce the following C++ code:

```
if (std::holds_alternative<Baz::Message_Text>(*message)) {
    auto text = std::get<Baz::Message_Text>(*message).value;
    std::cout << Baz::to_string(std::string("Got text: "));
    std::cout << Baz::to_string(text) << std::endl;
} else if (std::holds_alternative<Baz::Message_Number>(*message)) {
    auto num = std::get<Baz::Message_Number>(*message).value;
    std::cout << Baz::to_string(std::string("Got number: "));
    std::cout << Baz::to_string(num) << std::endl;
} else if (std::holds_alternative<Baz::Message_Empty>(*message)) {
    std::cout << Baz::to_string(std::string("Empty")) << std::endl;
}
```

In addition, as C++ does not support methods on variants, methods were implemented using normal functions where the first parameter is the enum itself. This provides the experience of accessing the “host object” as described by Abadi and Cardelli (1996). As with the variant structs, these were in the `Baz` namespace and prefixed by the enum’s name. For example, the `do_something` method in this Baz code:

```
enum Message {
    // ...

    fn do_something(a: int): int {
        return a + 12;
    }
}

fn main(): void {
    let message: Message = Message::Text("hi");
    let result: int = message.do_something(2);
}
```

will generate the following C++ code:

```
// ...

namespace Baz {
    int Message_do_something(Message *baz_this, int a) {
        return (a + 12);
    }
}

int main() {
    Message *message = (new Message(Baz::Message_Text{std::string("hi")}));
    int result = (Baz::Message_do_something(message, 2));
}
```

3.3.3. Optional Chaining

Optional chaining presented another difficulty during implementation.

```
let message: Message? = Message::Number(5);

// Optional chaining using `?.` syntax
let result: int? = message?.do_something(12);
```

Here, if `message` is `null`, then `do_something` will not be executed, and instead, the expression will evaluate to `null`. However, if `message` has a value, it will call `do_something` and return the result of that. At first thought, this could be implemented in C++ using the ternary operator:

```
std::optional<Message *> message = (new Message(Baz::Message_Number{5}));
std::optional<int> result = message.has_value() ?
message.value().do_something() : std::nullopt;
```

However, `message` can be any expression, including one that has side effects. Consider the following example:

```
fn get_message(): Message? {
    // Sometimes returns Message, sometimes `null`
}

fn main(): void {
    let result: int? = get_message()?.do_something(12);
}
```

Using the naive approach, the C++ code generated would be:

```
std::optional<int> result = get_message().has_value()
    ? std::optional{Baz::Message_do_something(get_message().value(), 12)}
    : std::nullopt;
```

This first checks if `get_message()` returns a value. If it does, then it calls `get_message()` again for the resulting value. The problem with this is that the second call could return a different value. To fix this, the transpiler needs to store the result in a variable for reuse. The problem is that this expression could be nested inside another expression, and due to the way my transpiler is implemented, it would be very difficult to work back out to create a variable. Therefore, the best solution was to use a new scope with a temporary variable:

```
std::optional<int> result = ({
    auto temp = (get_message());
    temp.has_value()
        ? std::optional{Baz::Message_do_something(temp.value(), 12)}
        : std::nullopt;
});
```

Another layer of complexity comes with combining this with methods on enums, as this is an extra place the transpiler has to check whether the method is being called on an enum, and generate the namespaced method call instead if so.

3.3.4. Arbitrary Ordering of User-Defined Types

In Baz, I wanted to support arbitrary ordering of structs and enums, as well as recursive and mutually recursive data structures. C++ does not support this out of the box - all structs must be declared lexically before use. To get around this problem, the first step of the C++ generator declares all structs at the top. Following this, enums are declared and defined. The definitions of the structs are generated after this - at which point everything has been declared, so the C++ compiler will be happy.

For example, linked lists are a mutually recursive data structure. Whilst optional types may be a simpler way to express this in Baz, for demonstration purposes, this implementation is based on Lisp lists, using cons and nil:

```
struct ConsData {
    value: int;
    next: ListNode;
}

// Each node either has a value and next node or is the end
enum ListNode {
    Cons(ConsData);
    Nil;
}
```

In this code, the `ListNode` enum has a `Cons` variant that contains a `ConsData` struct, which contains a `ListNode` enum. This is a mutually recursive data structure. Because of this, whichever way round we define the data structures, one will be referencing a type that is lexically after it. By declaring the structs at the start, the C++ code generator will produce the following code:

```
// Declarations of all structs
struct ConsData;

namespace Baz {
    struct ListNode_Cons;
    struct ListNode_Nil;
}

// Declarations of all enums
using ListNode = std::variant<Baz::ListNode_Cons, Baz::ListNode_Nil>;

// Definitions of structs - all types are declared at this point
struct ConsData {
    public:
        int value;
        ListNode *next;
};

namespace Baz {
    struct ListNode_Cons { ConsData *value; };
    struct ListNode_Nil {};
}
```

This approach does not currently support enums that reference each other, as they may not necessarily be declared in the order that C++ would expect. C++ does not support forward

declarations of enums, so this would require redesigning the implementation of enums. Due to the limited use cases of mutually recursive enum types, it would be a large investment for a small gain, and I decided this was out of the scope of the project.

3.3.5. Pattern Matching

Pattern matching can be performed on several data types, each of which has its own possible patterns. For this reason, I used a `MatchPattern` variant in C++ to represent any kind of pattern. The variants that Baz currently supports are enum patterns, catch-all patterns and null values. In the future, this could support many other patterns including concrete values, ranges of values, and individual properties on structs.

Pattern matching is exhaustive in Baz, meaning that every case has to be covered for it to compile. Type-checking is handled differently depending on the type of the expression that is being matched. If it is an enum, it will ensure that all variants of the enum have an associated branch. If it is an optional type, it will ensure that both the null and non-null branches have been covered. If it is an optional enum, it will ensure that all variants are covered, along with a final null branch.

Currently, the type checker does not support more complex match statements. For example, it is not currently possible to match on a subset of enum variants, with a final catch-all branch for other cases. This is because the exhaustiveness check is currently implemented by checking a 1 to 1 correspondence of branches and enum variants. This could be altered in the future to make match statements much more flexible.

3.4. System Testing

To ensure the scanner and parser worked as expected, I wrote automated tests using GoogleTest (Google, 2025). These were written as unit tests, so they were not dependent on each other. I mocked the scanner class to be able to test the parser without dependence on the real scanner.

I also wrote some automated tests for the type checker - making sure it succeeded when the types were correct, and failed when they were not. This was done as an integration test, as the type checker is inherently tied to the other phases of the transpiler. For example, the resolver will annotate the AST with essential information for the type checker. By performing an integration test, I ensure that the separate phases of the transpiler work together as expected.

These tests helped me identify some incorrect logic and unhandled cases and ensured that when I made changes, I could be confident in the correctness of the code. Some examples of these automated tests can be seen in Appendix B.

To check that the outputted C++ code ran as expected, I used a more manual approach - writing example programs in Baz, then manually transpiling and running them, followed by checking the output was correct. This allowed me to test more realistic code, see how the language features interact with each other, and ensure the implementation worked from end to end.

When the transpiler produced unexpected results or failed to compile, I used a C++ debugger to step through the compiler line by line and find where the code path deviated from what I expected. This made it really easy to locate the source of the issue and solve the problem quickly. It was also incredibly useful when the program produced a segfault without any extra information about where in the code or why it segfaulted.

A key bug I discovered when stepping through the debugger is that the results of the type checker were not properly being set on the AST, which meant that future steps (such as the C++ generator) did not have the information they needed to work correctly. Sometimes this would cause the transpiler to error, and sometimes it would silently continue without reporting an error. I have now

solved this by properly assigning a type to all expressions, and providing a getter method on each expression that first checks its type has been set correctly before it is returned. If the type has not been set, the program will crash and report the bug, thus removing the possibility of a silent error.

4. Investigation and Results

To evaluate the effectiveness of Baz, I attempted to implement some example programs, making note of areas it excels and falls short. The example programs can be seen in Appendix A. I was able to represent several data structures easily including linked lists, expression trees and binary trees. In comparison, implementing a linked list in Rust is not a trivial task (*Introduction - Learning Rust With Entirely Too Many Linked Lists*, 2025). In addition, the successful implementation of a Turing machine, shown in Appendix A.6, proves the Turing completeness of Baz. The type system managed to catch some mistakes I had made, without preventing me from representing data the way that I imagined it.

4.1. Example Type Errors

To assess the usefulness of Baz's type system, I deliberately introduced errors into sample programs to see whether the type checker would catch them at compile time. Baz was successful in identifying several classes of common mistakes.

Figure 6 demonstrates that the Baz type checker is able to ensure operations, such as arithmetic operators will only work on valid types. In this case, attempting a subtraction of an integer from a string does not make sense, so the Baz transpiler reports an error.

```
fn main(): void {  
    let foo: int = 42;  
    let bar: str = "hello world";  
  
    println(bar - foo);  
}
```

(a) Baz code

```
[line 5] Type error at '-': Operands  
must be the same type, or coercible  
to the same type.
```

(b) Output from the Baz transpiler

Figure 6: Attempting to subtract an integer from a string

Baz will also ensure that when pattern matching all possible values are covered. For example, when pattern matching on an enum, all variants of the enum must be covered in the match branches.

Figure 7 demonstrates that Baz is able to catch a missing branch when matching on an enum.

```
enum Foo {  
    Foo1;  
    Foo2;  
}  
  
fn main(): void {  
    let foo: Foo = Foo::Foo1;  
    match (foo) {  
        Foo::Foo1: {  
            println("handled");  
        }  
    }  
}
```

(a) Baz code

```
[line 8] Type error at 'match': Not  
all variants covered in pattern  
matching.
```

(b) Output from the Baz transpiler

Figure 7: Pattern matching without covering all possible values in the branches

In addition, Baz checks that the type of the variable being matched on is the same as the type of the branch patterns. In Figure 8, the Baz transpiler errors as `foo` is of type `Foo`, but the match branches are of type `Bar`.

```
enum Foo {
    Foo1;
    Foo2;
}

enum Bar {
    Bar1;
    Bar2;
}

fn main(): void {
    let foo: Foo = Foo::Foo1;
    match (foo) {
        Bar::Bar1: {
            println("handled");
        },
        Bar::Bar2: {
            println("handled");
        },
    }
}
```

(a) Baz code

```
[line 14] Type error at 'Bar': Match
pattern must be for the same enum
type.
```

(b) Output from the Baz transpiler

Figure 8: Pattern matching where branch type is different from variable type

Another common mistake is accessing a property that does not exist on a struct. This could happen due to something as simple as a typo, but could also be due to thinking the variable is another type, or forgetting to update a part of code during a refactor. Figure 9 demonstrates this - `prop2` is trying to be accessed on `foo`, which does not have a property called `prop2`, and therefore an error is reported.

```
struct Foo {
    prop1: int;
}

fn main(): void {
    let foo: Foo = Foo{ prop1: 5 };
    foo.prop2;
}
```

(a) Baz code

```
[line 7] Resolving error at 'prop2':
Could not find member on struct.
```

(b) Output from the Baz transpiler

Figure 9: Accessing a property that does not exist on a struct

4.2. Areas for Improvement

While writing the code, however, I discovered some parts that were tedious to write. The first of which is that enum payloads currently only support one piece of data. This means that for a tuple payload, you would need to define a new struct with two properties just for that enum variant. This becomes verbose and cumbersome, especially when you have several enum variants, each with different data. To improve this, Baz could be extended to support enums having multiple pieces of data, similar to Rust's enums.

The second tedious area that I found was “unwrapping” an optional type. While it is straightforward to check for a null value, it is not as easy to get the non-optional type out after asserting that it is not `null`. Currently, this can be achieved in several ways, but the two most straightforward are using pattern matching to bind the non-null case to a variable, and using optional chaining and fallbacks. The issue here is that pattern matching requires specifying both branches (the null and non-null case), which sometimes is not necessary and causes deep nesting, and optional chaining is redundant after a `null` check. To improve this, Baz could support syntax similar to Zig, where the non-optional value can be bound in an if statement:

```
const optional: ?i32 = 42;
if (optional) |value| {
    std.debug.print("Got a value: {}\n", .{value});
}
```

The last area that I noticed could be improved was error handling. This was not explored deeply due to the scope of the project, but with the richness of the type system, it is possible to manually create an error handling system with a Result enum:

```
enum Result {
    Ok(int),
    Error(str),
}
```

This allows proper handling of errors. However, error handling is very common, and I believe that the current pattern-matching support in Baz is not yet flexible enough to make handling these errors elegant. For example, if you had two `Result`s, and wanted to pattern match both, it would have to be done in a nested `match` statement. If Baz were to support multiple target values in a `match` statement, as seen in Listing 4, these could be checked at the same time, reducing the nesting. Furthermore, a common scenario when handling an error is to return early if an error is received. Supporting syntax for an early return would help simplify the code, reducing the boilerplate of error handling.

Finally, the addition of an array/list data type would eliminate the need to implement a linked list for each data type, thus simplifying code, and removing redundancies.

4.3. Performance and Scalability

While not a key focus when developing Baz, it is interesting to evaluate the performance and scalability. Baz performs transpilation in linear time. The scanner is linear in terms of the number of characters in the source code, the parser is linear in terms of the number of tokens, and all further stages are linear in terms of the number of expressions/statements. Both the number of tokens and expressions/statements is a linear scaling of the number of characters. Due to this, transpile times are very fast. Appendix A reports the transpilation time of each program. These were all below 5ms.

Unfortunately, as Baz transpiles to C++, there is the extra step of compiling the C++ code. This means all compilations will be at least as slow as C++ compilations. In addition, no optimisations are performed to generate efficient C++ code, meaning that the output binary will likely be at least as slow as an equivalent program written by an experienced C++ developer.

The current lack of support for multiple files and packages would make it difficult to develop and maintain larger programs in Baz.

```

// Enum to represent a successful operation with a value, or an error with an
// error message
enum Result {
    Ok(int);
    Err(str);
}

fn main(): void {
    let l: Result = Result::Ok(5);
    let r: Result = Result::Err("failed");

    // Divide l by r if neither are errors, and r is not zero
    match (l, r) {
        (Result::Ok(_), Result::Ok(0)): {
            println("division by zero");
        },
        (Result::Ok(a), Result::Ok(b)): {
            println(a / b);
        },
        (Result::Err(e), _): {
            println(e)
        },
        (_, Result::Err(e)): {
            println(e)
        },
    }
}

```

Listing 4: Pattern matching on multiple values

5. Critical Appraisal & Conclusions

Throughout this project, I have encountered some interesting challenges, which have given me a new understanding and respect for the complexities of language design. For example, I have found it particularly challenging to balance simplicity, intuitiveness, and expressiveness. In Section 3.2 and Section 3.3, I discuss the cases where I had to make compromises in terms of the language design.

5.1. Successes

This project involved the design and implementation of a fully functional programming language, including type checking of an expressive algebraic type system, safe nullable types, and pattern matching. The whole transpiler pipeline was coded from scratch in C++, making the architecture flexible.

I have been able to test the language with a variety of programs that demonstrate its core features, as seen in Appendix A. The type checker has succeeded in catching several mistakes I made when developing these programs. In addition, the prototype tree walk program in Listing 1 that I designed at the start is now a compilable program that behaves as expected, after a minor modification to use the new `::` syntax for enums. A key achievement with Baz was showing that it is Turing complete, as evidenced by the Turing machine implementation in Appendix A.6.

The flexibility of the type system also allows for the creation of a `Result` type for error handling. The current ergonomics around this could be improved, but it shows that this form of error handling is achievable.

5.2. Trade-Offs

Traits and generics were a feature I originally aimed to implement at the start of the project, but it ended up being out of the scope of this project. This advanced feature would have enabled a lot more flexibility in the type system, however, I decided to focus on implementing pattern matching instead, which I believed to be more fundamental to the language. Support for these features would massively improve the flexibility of the type system. For example, it would enable a generic `Result<T, U>` type, similar to Rust's `Result` type, that supports specifying the successful type and the error type. This can then be reused in multiple contexts without having to create a new enum for each type.

As discussed in Section 3.3.2, a compromise I made in the language design was changing `Enum.variant` to `Enum::variant`, introducing more syntax. This was unideal, but made the implementation a lot easier - avoiding having to check every time `.` is used whether it is an enum variant, or getting a property on a variable. With more time, it may have been possible to make changes to the architecture of the transpiler to make the original syntax possible without the added complexity in several areas of the transpiler pipeline.

In addition, as discussed in Section 3.2.1, I tried to aim for consistency between function parameters and pattern-matching bound variables, but this ended up forcing the compromise of the `this` object being passed by reference. Therefore the `this` object is not consistent with other method parameters. An alternative approach would be to have a specific syntax for binding by reference such as `Enum::Variant(&bound_variable)`, which would support reassigning the bound variable and having that update the match target. I originally decided against this due to it forcing the programmer to understand pass-by-reference vs pass-by-value, but it may be sufficient to provide a clear compile-time warning/error if the programmer tries to reassign a pass-by-value variable.

5.3. Future Work

The success of this prototype of Baz demonstrates that it is possible to have an expressive type system within an accessible and easy-to-use language. Due to this, there are many possible areas for extension that were not originally considered due to the scope of this project.

Transpiling to C++ allowed me to get a proof of concept working quickly, but is more suited to a prototype. It requires there to be a direct translation from code in Baz to code in C++, which is not always trivial, as demonstrated by the implementation of both enums and optional chaining discussed in Section 3.3. To ease the implementation of more complex features, that may not be non-trivial to implement in C++, different back-end options could be explored. LLVM would be a good candidate as it would retain the benefits of transpiling to C++ - the support on most platforms, and optimisations provided by the LLVM compiler.

With a more fleshed-out back end, it would also be feasible to implement a runtime and garbage collector, which would allow for larger programs to be written without worrying about the current memory leaks. If compilation speeds are of interest, it would also be possible to implement incremental compilation or caching to speed up subsequent compilations and allow fast iteration. Furthermore, there could be optimisations performed on the LLVM code to make the end program more efficient.

Pattern matching is another area that could be extended. For example, supporting multiple target values would enable simultaneously handling multiple variables, without needing nested match statements. This would reduce boilerplate code and increase readability, allowing for the scenarios discussed in Section 4.2. It could also be extended to support more patterns. At the moment, it is only possible to match on an enum variant, a null value, or catch the remaining cases. To improve flexibility, pattern matching could have support for concrete values and ranges of values. Another

```

enum Foo {
    Variant(Bar);
}

struct Bar {
    prop1: int;
    prop2: str;
}

fn main(): void {
    let foo: Foo = Foo::Variant(Bar {
        prop1: 1234,
        prop2: "some_value",
    });

    match (foo) {
        Foo::Variant(Bar { prop1: value, prop2: "some_value" }): {
            // Use `value` bound from `prop1`
        },
        // Do nothing in all other cases
        _: {},
    }
}

```

Listing 5: Pattern matching on nested data types

interesting and useful avenue to explore would be pattern matching on structs, as well as nested data structures, as this could reduce the need for nested match statements for more complex data structures. Listing 5 shows how these extensions to pattern matching could be used.

Another potential area of extension is multiple payloads within an enum variant. Currently, enum variants can only have one payload, however, it is a very common use case to require a tuple of values in a payload. Without support for multiple payloads, this has to be implemented using an auxiliary struct. By supporting multiple payloads, it would remove the need to create these auxiliary structs, and thus make enums more convenient by reducing boilerplate code, as shown in Listing 6.

As discussed in Section 4.2, the addition of a built-in array/list data type would simplify code by removing the need to create a linked list by hand.

Support for multiple source files would also be very useful in structuring and maintaining larger programs in Baz. This would likely be linked with support for packages and namespacing and would require significant work to resolve dependencies and safeguard against circular imports.

Additionally, adding a standard library with I/O functions would elevate the real-world capabilities of Baz.

```

enum Foo {
    Variant(int, str);
}

fn main(): void {
    let foo: Foo = Foo::Variant(42, "hello");
}

```

Listing 6: Multiple payloads on an enum variant

Furthermore, while the transpiler provides safety checks through the type checker, these rules have not been formally defined yet. It would be possible to formalise them into a rigorous type system, as discussed in Section 3.2.2, which could be used to provide guarantees about how the system will behave. Rust has followed this path - while the creators did not formally define the type system when creating the language, work has been done since by Jung et al. (2017) to formally prove its guarantees.

Finally, a long-term goal could be to bootstrap the Baz compiler, so that the compiler is written in Baz itself. This would be an interesting exercise to test the capabilities and limitations of Baz in a real-world, large-scale project.

5.4. Final Remarks

In this project, I set out to design and implement a programming language that balances accessibility with an expressive type system. I believe it achieves this goal: Baz is a fully functioning language that feels accessible and lightweight, whilst still supporting a rich type system with features such as algebraic data types, pattern matching, and optional types. Together, these offer safety and expressive power without overwhelming complexity.

Word count: 10105

6. Glossary

Term	Definition
AST	An Abstract Syntax Tree (AST) is a representation of a program in a tree structure. It contains all the information about the program in a more abstract form, which is easier to perform analysis on.
Algebraic Data Types	A composite data type, constructed by combining other types. This includes sum types and product types.
Bootstrapping	The process of writing a compiler for a language in that same language. It starts with a compiler written in an existing language, which is then used to compile a new version of the compiler written in the target language itself.
Compiler	A program that turns high-level source code into a lower-level form, such as machine code.
Destructuring	A language feature that enables the unpacking values from composite data structures using a concise, declarative syntax. Commonly unpacked values include enum payloads, struct properties, and array elements.
Enum	A data type that defines a set of named variants which may hold a payload of associated data. A variable can only be one of the variants at a time. This is an example of a sum data type.
Garbage Collection	Automatic memory management where the runtime will work out which memory may be being used at a given time, and automatically free any memory that cannot be used. This is an alternative to reference counting.
Interpreter	A program that takes high-level source code and runs it directly, without turning it into machine code first.
Lexeme	The raw string value of a token.
Match Exhaustiveness	Checking that all possible cases of a value have been covered in the branches of pattern matching.
Optional Chaining	A language feature that allows safe access to a property on an optional object by only performing the lookup if the object is not null. For example <code>foo?.bar</code> will only attempt to get the property <code>bar</code> if <code>foo</code> is not null.
Pass Object Reference by Value	The reference to an object is passed to a function by copying the reference. This copied reference will still refer to the same object in memory, but is distinct from the original reference.
Pattern Matching	A language feature that allows checking a value against a pattern and extracting data from within the object. Patterns are constructs that represent a subset of values of a type. For an enum this could be a variant, for a struct this could be a property with a specific value, etc.
Product Data Types	A data type where several values can be combined together to make a compound type. In Baz, this is represented using structs.
Reference Counting	Automatic memory management where the runtime will keep track of how many references there are to each value. If this reaches 0 at any point, that memory is freed as it cannot be used again. This is an alternative to garbage collection.
Resolver	A phase in the compiler that links variable uses to their declarations. This allows gathering of extra information such as whether the variable exists and what type it is.

Term	Definition
Runtime	A sub-program that runs alongside the programmers's program, providing extra functionality such as automatic memory management, I/O, etc.
Static Typing	A system where the types of variables are able to be determined and checked at compile time.
Strangess Budget	A term coined by Klabnik (2015) referring to the amount of novelty (e.g. new features) a programmer is willing to learn when trying out a new language.
Struct	A data type that collects multiple other data types together into a compound type. For example, grouping together related data such as the name and age of a person. This is an example of a product data type.
Sum Data Types	A data type where the value is one out of a set of types. In Baz, this is represented using enums.
Token	A basic unit of the programming language, often being a single word or symbol. For example, a symbol such as <code>>=</code> , a variable name, a literal value, etc. It contains data such as the type of token, the lexeme, and the line number that the token came from.
Trait/Interface	A programming construct that allows the programmer to define a set of method signatures that must be implemented on a type. This allows for decoupling of the implementation details from its use.
Transpiler	A program that takes high-level source code and turns it into source code in another high-level language.
Type Coercion	Automatic conversion from one type to another. Usually only done when the conversion is obvious e.g. turning an integer into float.
Type System	A set of rules on the types of variables that defines which programs are valid and which are not.

Bibliography

- Abadi, M. and Cardelli, L., 1996. *A theory of objects*. 1st ed. 1996. Monographs in Computer Science. New York, New York State: Springer. Publication Title: A theory of objects.
- Aho, A.V., 2014. *Compilers : principles, techniques, and tools*. Pearson new international, second edition. Always learning. Essex, England: Pearson. Publication Title: Compilers : principles, techniques, and tools.
- Albrizio, G., 2023. *Rust Powerful Type System* [Online]. Available from: <https://medium.com/bridging-the-gap-between-node-js-and-rust/rust-powerful-type-system-54b9e32b7425> [Accessed 6 December 2024].
- Bello, A., 2024. *Advantages of Static Typing* [Online]. Available from: <https://www.amorserv.com/insights/advantages-of-static-typing>.
- Bolognese, L., 2012. *Exceptions vs. Return Values to represent errors (in F#) – Conceptual view - Luca Bolognese* [Online]. Available from: <https://www.lucabol.com/posts/2012-11-19-exceptions-vs-return-values-to-represent-errors-in-f-i-conceptual-view> [Accessed 15 April 2025].
- Boyle, E., 2019. *Static Types vs Dynamic Types. Stop fighting and make my life easier already* [Online]. Available from: <https://instil.co/blog/static-vs-dynamic-types/> [Accessed 17 November 2024].
- Chiusano, P., 2016. *The Advantages of Static Typing, Simply Stated* [Online]. Available from: <https://pchiusano.github.io/2016-09-15/static-vs-dynamic.html>.
- Cooper, K. and Torczon, L., 2012. *Engineering a compiler*. 2nd ed. Amsterdam: Elsevier. Publication Title: Engineering a compiler.
- Dieterichs, H., 2017. *TypeScript's Type System is Turing Complete · Issue #14833 · microsoft/TypeScript* [Online]. Available from: <https://github.com/microsoft/TypeScript/issues/14833> [Accessed 6 December 2024].
- Dudeja, K., 2017. *The Beauty of Go* [Online]. Available from: <https://medium.com/hackernoon/the-beauty-of-go-98057e3f0a7d>.
- Elsinga, R.J., 2020. *The impact of migrating from PHP to Golang* [Online]. Available from: <https://roelo.fjanelsinga.com/articles/the-impact-of-migrating-from-php-to-golang> [Accessed 24 November 2024].
- Endrikat, S., Hanenberg, S., Robbes, R. and Stefik, A., 2014. *How do API documentation and static typing affect API usability?* [Online]. New York, NY, USA: ACM, pp.632–642. Book Title: Proceedings of the 36th International Conference on Software Engineering. Available from: <https://doi.org/10.1145/2568225.2568299>.
- Farrer, E., 2012. *Unit Testing Isn't Enough. you Need Static Typing Too* [Online]. Available from: <http://evanfarrer.blogspot.com/2012/06/unit-testing-isnt-enough-you-need.html>.
- Fiala, J., Itzhaky, S., Müller, P., Polikarpova, N. and Sergey, I., 2023. Leveraging Rust Types for Program Synthesis. *Reproduction Package for Article "Leveraging Rust Types for Program Synthesis"* [Online], 7(PLDI), pp.1414–1437. Available from: <https://doi.org/10.1145/3591278> [Accessed 26 November 2024].
- Google, 2025. *google/googletest* [Online]. Google. original-date: 2015-07-28T15:07:53Z. Available from: <https://github.com/google/googletest> [Accessed 14 April 2025].
- Hanenberg, S., Kleinschmager, S., Robbes, R., Tanter, É. and Stefik, A., 2013. An empirical study on the impact of static typing on software maintainability. *Empirical Software Engineering* [Online], 19. Available from: <https://doi.org/10.1007/s10664-013-9289-1>.

Hoare, T., 2009. *Null References: The Billion Dollar Mistake* [Online]. Available from: <https://www.infoq.com/presentations/Null-References-The-Billion-Dollar-Mistake-Tony-Hoare/> [Accessed 16 November 2024].

How has learning and working in Rust influenced how you think about writing software? - community [Online], 2020. Section: community. Available from: <https://users.rust-lang.org/t/how-has-learning-and-working-in-rust-influenced-how-you-think-about-writing-software/42836> [Accessed 6 December 2024].

How to write documentation - The rustdoc book [Online], 2024. Available from: <https://doc.rust-lang.org/rustdoc/how-to-write-documentation.html> [Accessed 5 December 2024].

Hudak, P. and Fasel, J.H., 1992. A gentle introduction to Haskell. *SIGPLAN notices* [Online], 27(5), pp.1–52. Publisher: Association for Computing Machinery. Available from: <https://doi.org/10.1145/130697.130698>.

Influences - The Rust Reference [Online], 2024. Available from: <https://doc.rust-lang.org/reference/influences.html> [Accessed 1 December 2024].

Introduction - Learning Rust With Entirely Too Many Linked Lists [Online], 2025. Available from: <https://rust-unofficial.github.io/too-many-lists/> [Accessed 28 April 2025].

JetBrains, 2020. *Python Developers Survey 2020 Results* [Online]. Available from: <https://www.jetbrains.com/lp/python-developers-survey-2020> [Accessed 26 November 2024].

Jintin, 2024. *The Beauty of Kotlin Type System* [Online]. Available from: <https://proandroiddev.com/the-beauty-of-kotlin-type-system-25d0a187b130> [Accessed 17 November 2024].

Jung, R., Jourdan, J.-H., Krebbers, R. and Dreyer, D., 2017. RustBelt: securing the foundations of the Rust programming language. *Proc. ACM Program. Lang.* [Online], 2(POPL). Place: New York, NY, USA Publisher: Association for Computing Machinery. Available from: <https://doi.org/10.1145/3158154>.

Kechagia, M. and Spinellis, D., 2014. *Undocumented and unchecked: exceptions that spell trouble* [Online]. New York, NY, USA: ACM, pp.312–315. Book Title: Proceedings of the 11th Working Conference on Mining Software Repositories. Available from: <https://doi.org/10.1145/2597073.2597089>.

Klabnik, S., 2015. *The language strangeness budget* [Online]. Available from: <https://steveklabnik.com/writing/the-language-strangeness-budget> [Accessed 30 November 2024].

Linnington, C., 2024. *Stop using Exceptions* [Online]. Available from: <https://medium.com/@no1.melman10/stop-using-exceptions-4a72b9ab337f> [Accessed 6 December 2024].

Maguire, S., 2015. *How I Learned to Stop Worrying and Love the Type System :: Reasonably Polymorphic* [Online]. Available from: <https://reasonablypolymorphic.com/blog/love-types/index.html> [Accessed 6 December 2024].

Mayer, C., Hanenberg, S., Robbes, R., Tanter, É. and Stefik, A., 2012. An empirical study of the influence of static type systems on the usability of undocumented software. *SIGPLAN notices* [Online], 47(10), pp.683–702. Publisher: Association for Computing Machinery. Available from: <https://doi.org/10.1145/2398857.2384666>.

Milner, R., 1978. A theory of type polymorphism in programming. *Journal of computer and system sciences* [Online], 17(3), pp.348–375. Publisher: Elsevier Inc. Available from: [https://doi.org/10.1016/0022-0000\(78\)90014-4](https://doi.org/10.1016/0022-0000(78)90014-4).

Mo, J., 2023. *Why Type Safety is Important* [Online]. Available from: <https://www.shuttle.dev/blog/2023/11/29/type-safety> [Accessed 17 November 2024].

Moskala, M., 2023. *The beauty of Kotlin type system* [Online]. Available from: https://kt.academy/article/kfde-type_system [Accessed 6 December 2024].

Murzin, S., Park, M., Sankel, D., Sarginson, D. and Stroustrup, B., 2019. *Pattern Matching*. *Newbie learning how to deal with the borrow checker - help* [Online], 2020. Section: help. Available from: <https://users.rust-lang.org/t/newbie-learning-how-to-deal-with-the-borrow-checker/40972> [Accessed 24 November 2024].

Nystrom, R., 2021. *Crafting interpreters*. Genever Benning.

Oh, W. and Oh, H., 2022. PyTER: effective program repair for Python type errors. *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* [Online]. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, pp.922–934. Available from: <https://doi.org/10.1145/3540250.3549130> [Accessed 26 November 2024].

Oh, W. and Oh, H., 2024. Towards Effective Static Type-Error Detection for Python. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* [Online]. ASE '24. New York, NY, USA: Association for Computing Machinery, pp.1808–1820. Available from: <https://doi.org/10.1145/3691620.3695545> [Accessed 26 November 2024].

OpenAPI Tools, 2024. *OpenAPITools/openapi-generator* [Online]. OpenAPI Tools. original-date: 2018-05-12T09:57:56Z. Available from: <https://github.com/OpenAPITools/openapi-generator> [Accessed 5 December 2024].

Oracle, n.d. *Optional (Java Platform SE 8)* [Online]. Available from: <https://docs.oracle.com/javase/8/docs/api/java/util/Optional.html> [Accessed 15 April 2025].

Ortiz, B., 2020. 🦀 *Rust Reviewed: Is the hype justified?* 🦀 [Online]. Available from: <https://dev.to/somedood/rust-reviewed-is-the-hype-justified-1pa1> [Accessed 1 December 2024].

The PHP Documentation Group, 2019. *PHP: New Features - Manual* [Online]. Available from: <https://www.php.net/manual/en/migration74.new-features.php> [Accessed 2 December 2024].

Pierce, B.C., 2002. *Types and programming languages*. Cambridge, Massachusetts: MIT Press. Publication Title: Types and programming languages.

Pistunovich, N., Boyle, M. and Shepherd, C., 2024. *Migrating from PHP to Go with Matthew Boyle & Chris Shepherd (Go Time #316)* [Online]. Available from: <https://changelog.com/gotime/316> [Accessed 24 November 2024].

Pitt, C., 2016. *Typed PHP: Stronger Types for Cleaner Code* [Online]. 1st ed. 2017. Berkeley, CA: Apress L. P. Available from: <https://doi.org/10.1007/978-1-4842-2114-3>.

Pádua, G.B.d. and Shang, W., 2017. *Studying the Prevalence of Exception Handling Anti-Patterns* [Online]. arXiv. arXiv:1704.00778 [cs]. Available from: <https://doi.org/10.48550/arXiv.1704.00778> [Accessed 15 April 2025].

Raschka, S., Patterson, J. and Nolet, C., 2020. Machine Learning in Python: Main Developments and Technology Trends in Data Science, Machine Learning, and Artificial Intelligence. *Information* [Online], 11(4). Publisher: Multidisciplinary Digital Publishing Institute. Available from: <https://doi.org/10.3390/info11040193> [Accessed 6 December 2024].

- Rathi, M., 2020. *An accessible introduction to type theory and implementing a type-checker — mukulrathi.com* [Online]. Available from: <https://mukulrathi.com/create-your-own-programming-language/intro-to-type-checking/> [Accessed 7 November 2024].
- Renoth, P., 2021. *Rust ownership and borrows - Fighting the borrow-checker* [Online]. Available from: <https://dev.to/daaitch/rust-ownership-and-borrows-fighting-the-borrow-checker-4ea3> [Accessed 24 November 2024].
- Riston, W., 2017. *Why is Haskell so hard to learn?* [Online]. Available from: <https://www.quora.com/Why-is-Haskell-so-hard-to-learn> [Accessed 24 November 2024].
- The Rust Survey Team, 2020. *Rust Survey 2020 Results | Rust Blog* [Online]. Available from: <https://blog.rust-lang.org/2020/12/16/rust-survey-2020.html#improved-learnability> [Accessed 1 December 2024].
- Sagonas, K. and Luna, D., 2008. Gradual typing of erlang programs: a wrangler experience. *Proc. 7th ACM SIGPLAN Workshop on ERLANG*. New York, NY, USA: ACM, pp.73–82.
- Scott, M.L., 2009. *Programming language pragmatics*. 3rd ed. Amsterdam ;: Elsevier/Morgan Kaufmann Pub. Publication Title: Programming language pragmatics.
- Shamrell-Harrington, N., 2022. *The Rust Borrow Checker - a Deep Dive* [Online]. Available from: <https://www.infoq.com/presentations/rust-borrow-checker/> [Accessed 2 December 2024].
- Shui, Y., n.d. *Lifetime woes Pt.2 - The Adventure of Creating a Wayland Compositor Toolbox in Rust* [Online]. Available from: <https://runa.yshui.dev/lifetime-p2.html> [Accessed 24 November 2024].
- Simsek, M., 2021. *Our Journey Moving from PHP to Go* [Online]. Available from: <https://medium.com/beyn-technology/our-journey-moving-from-php-to-go-4b4513f92e75> [Accessed 24 November 2024].
- Sitnikovski, B., 2023. *Introduction to Dependent Types with Idris: Encoding Program Proofs in Types* [Online]. 1st ed. Berkeley, CA: Apress. Available from: <https://doi.org/10.1007/978-1-4842-9259-4>.
- Stack Overflow, 2020. *Stack Overflow Developer Survey 2020* [Online]. Available from: https://survey.stackoverflow.co/2020?utm_source=social-share&utm_medium=social&utm_campaign=dev-survey-2020 [Accessed 24 November 2024].
- Stack Overflow, 2024. *Technology | 2024 Stack Overflow Developer Survey* [Online]. Available from: <https://survey.stackoverflow.co/2024/technology#most-popular-technologies-language> [Accessed 24 November 2024].
- Steimann, F. and Mayer, P., 2005. Patterns of Interface-Based Programming. *Journal of object technology* [Online], 4(5). Publisher: EtH Zurich. Available from: <https://doi.org/10.5381/jot.2005.4.5.a1>.
- Stuchlik, A. and Hanenberg, S., 2011. Static vs. dynamic type systems: an empirical study about the relationship between type casts and development time. *Proceedings of the 7th symposium on Dynamic languages* [Online]. DLS '11. New York, NY, USA: Association for Computing Machinery, pp.97–106. Available from: <https://doi.org/10.1145/2047849.2047861> [Accessed 26 November 2024].
- Tietz-Sokolsaya, N., 2023. *Why Rust's Learning Curve Seems Harsh, and Ideas to Reduce It* [Online]. Available from: <https://ntietz.com/blog/rust-resources-learning-curve/> [Accessed 17 November 2024].
- Vaner, M., 2018. *Why is Rust Difficult* [Online]. Available from: <https://vornier.github.io/difficult.html> [Accessed 17 November 2024].

The White House, 2024. *Back to the Building Blocks: A Path Towards Secure and Measurable Software* [Online]. Available from: <https://www.whitehouse.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf> [Accessed 6 December 2024].

Wilson, M., 2010. *Quality Matters: The Worst Form of 'Error' Handling Except For All The Others* [Online]. Available from: https://accu.org/journals/overload/18/98/wilson_1681/ [Accessed 15 April 2025].

Yakov Fain, 2020. *The Advantages of Using TypeScript vs JavaScript*. Manning Publications.

Yourdon, E. and Constantine, L.L., 1979. *Structured design : fundamentals of a discipline of computer program and systems design*. Englewood Cliffs ;; Prentice-Hall. Publication Title: Structured design : fundamentals of a discipline of computer program and systems design.

Appendix

A Example Programs

Each program was transpiled 15 times, and the average time to transpile was recorded.

A.1 Counter

A simple program to count up to a number.

```
fn count_to(limit: int): void {
  for (let i: int = 1; i <= limit; i = i + 1) {
    println(i);
  }
}

fn main(): void {
  println("Counting to 5:");
  count_to(5);
}
```

Average transpile time: 552 microseconds

Output:

```
Counting to 5:
1
2
3
4
5
```

A.2 Greeter

A simple program to greet a user.

```
struct Greeter {
  name: str;
  greeted: bool;

  // Say "Hello <name>!" the first time
  // Afterwards, say "We've already met, <name>."
  fn greet(): void {
    if (!this.greeted) {
      println("Hello, " + this.name + "!");
      this.greeted = true;
    } else {
      println("We've already met, " + this.name + ".");
    }
  }
}

fn main(): void {
  let g: Greeter = Greeter {
    name: "Ada",
    greeted: false,
```



```
};

g.greet(); // First greeting
g.greet(); // Should say already greeted
}
```

Average transpile time: 814 microseconds

Output:

```
Hello, Ada!
We've already met, Ada.
```

A.3 Binary Search Tree

A binary search tree which allows inserting, checking if a number is in the tree, and performing an in-order traversal.

```
struct TreeNode {
    value: int;

    // Optional left and right children
    left: TreeNode?;
    right: TreeNode?;

    // Print the tree in-order
    fn in_order_traversal(): void {
        this.left?.in_order_traversal();
        println(this.value);
        this.right?.in_order_traversal();
    }

    // Insert a number into the correct place in the tree
    fn insert(num: int): void {
        if (num < this.value) {
            // Insert into the left node, or create it if it does not exist
            if (this.left == null) {
                this.left = TreeNode { value: num, left: null, right: null };
            } else {
                this.left?.insert(num);
            }
        } else {
            // Insert into the right node, or create it if it does not exist
            if (this.right == null) {
                this.right = TreeNode { value: num, left: null, right: null };
            } else {
                this.right?.insert(num);
            }
        }
    }
}

// Check if the number exists in the tree
fn contains(num: int): bool {
    if (num == this.value) {
```

```

        return true;
    } else {
        if (num < this.value) {
            // Check left node. If no left node, the number is not in the tree
            return this.left?.contains(num) ?? false;
        } else {
            // Check right node. If no right node, the number is not in the tree
            return this.right?.contains(num) ?? false;
        }
    }
}
}
}

fn main(): void {
    // Set up tree and insert some values
    let tree: TreeNode = TreeNode {
        value: 5,
        left: null,
        right: null,
    };

    tree.insert(3);
    tree.insert(9);
    tree.insert(6);
    tree.insert(8);

    println("==== IN-ORDER TRAVERSAL ====");
    tree.in_order_traversal();

    println("\n==== NUMBERS IN TREE ====");
    let i: int = 0;
    while (i < 10) {
        print(i);
        print(": ");
        println(tree.contains(i));

        i = i + 1;
    }
}

```

Average transpile time: 1669 microseconds

Output:

```

==== IN-ORDER TRAVERSAL ====
3
5
6
8
9

==== NUMBERS IN TREE ====
0: false
1: false

```

```
2: false
3: true
4: false
5: true
6: true
7: false
8: true
9: true
```

A.4 Expression Tree

A tree data structure for evaluating arithmetic expressions.

```
// Struct to hold two expressions
struct ExprPair {
    left: Expr;
    right: Expr;
}

// Expression is either a number or one of the 4 basic arithmetic operations on
two sub-expressions
enum Expr {
    Number(float);

    Add(ExprPair);
    Subtract(ExprPair);
    Multiply(ExprPair);
    Divide(ExprPair);

    // Calculate the resulting value of this expression
    fn evaluate(): float {
        match (this) {
            // If it's a number, return that
            Expr::Number(value): { return value; },

            // If it's an operation, evaluate the sub-expressions and operate on those
            Expr::Add(pair): {
                return pair.left.evaluate() + pair.right.evaluate();
            },
            Expr::Subtract(pair): {
                return pair.left.evaluate() - pair.right.evaluate();
            },
            Expr::Multiply(pair): {
                return pair.left.evaluate() * pair.right.evaluate();
            },

            // Division requires extra care for the divide by 0 case
            Expr::Divide(pair): {
                let right: float = pair.right.evaluate();
                if (right == 0.0) {
                    panic("Cannot divide by 0");
                }

                return pair.left.evaluate() / pair.right.evaluate();
            }
        }
    }
}
```

```

    },
  }
}
}

fn main(): void {
  // (5.0 + (90.0 / 10.0)) * 8.0
  let expr: Expr = Expr::Multiply(ExprPair{
    left: Expr::Add(ExprPair{
      left: Expr::Number(5.0),
      right: Expr::Divide(ExprPair {
        left: Expr::Number(90.0),
        right: Expr::Number(10.0),
      }),
    }),
    right: Expr::Number(8.0),
  });

  // Evaluate and print result
  println(expr.evaluate());
}

```

Average transpile time: 1303 microseconds

Output:

112

A.5 Linked List

A linked list of integers. Includes an `append` method to add to the end of the list and an `insert_at` to insert at a particular position

```

// Each node in the list has a value, and an optional previous and next node
struct Node {
  value: int;
  next: Node?;
  prev: Node?;

  // Adds to the end of the list
  fn append(value: int): void {
    match (this.next) {
      // If at the end - add a new node to the end
      null: {
        this.next = Node {
          value: value,
          next: null,
          prev: this,
        };
      },
      // Otherwise recursively append to the next node
      next: {
        next.append(value);
      },
    }
  }
}

```

```

    }
}

// Insert a node at a specific position in the list. It will return the start
of the list
// If idx is 0, this will return the new node - the new start
fn insert_at(idx: int, value: int): Node {
    // If adding at start, return new node pointing to old first node
    if (idx == 0) {
        return Node {
            value: value,
            next: this,
            prev: this.prev,
        };
    }

    match (this.next) {
        // If no next node, insert at the end
        null: {
            this.next = Node {
                value: value,
                next: null,
                prev: this,
            };
        },
        // Otherwise, recursively insert_at with the tail of the list and (idx-1)
        next: {
            this.next = next.insert_at(idx - 1, value);
        },
    }

    return this;
}

// Prints list as "a, b, c, d\n"
fn print_list(): void {
    match (this.next) {
        null: {
            println(this.value);
        },
        next: {
            print(this.value);
            print(", ");
            next.print_list();
        }
    }
}

}

fn main(): void {
    // Create the first node
    let list: Node = Node {
        value: 4,
        next: null,
        prev: null,
    }
}

```

```

};

// Add to the end
list.append(5);
list.append(7);

// Insert other numbers into their correct positions
list = list.insert_at(0, 3);
list = list.insert_at(0, 1);
list = list.insert_at(1, 2);
list = list.insert_at(5, 6);

// Print the resulting list
list.print_list();
}

```

Average transpile time: 1447 microseconds

Output:

```
1, 2, 3, 4, 5, 6, 7
```

A.6 Turing Machine

A Turing machine implemented in Baz. It has a single tape, a set of states, and a set of transitions. The transitions define how to change the tape, where to move the head, and which state to move into next. This particular Turing machine inverts the value that is initially on the tape.

By implementing a Turing machine, I have shown that Baz is Turing complete.

```

struct TapeCell {
    val: str;

    // Doubly linked list
    prev: TapeCell?;
    next: TapeCell?;

    fn append(val: str): void {
        match (this.next) {
            null: { this.next = TapeCell { val: val, prev: null, next: null }; },
            next: { next.append(val); }
        }
    }

    // Gets the previous value if it exists, or creates a '#' cell to the left,
    and returns it
    // This creates the illusion of an infinite tape
    fn get_prev_or_extend_empty(): TapeCell {
        match (this.prev) {
            null: {
                let fallback_empty_tape_cell: TapeCell = TapeCell { val: "#",
prev: null, next: this };
                this.prev = fallback_empty_tape_cell;
                return fallback_empty_tape_cell;
            }
        }
    }
}

```

```

        },
        prev: {
            return prev;
        },
    }
}

// Gets the next value if it exists, or creates a '#' cell to the right, and
returns it
// This creates the illusion of an infinite tape
fn get_next_or_extend_empty(): TapeCell {
    match (this.next) {
        null: {
            // If no tape to the right - create a new '#' cell on the fly
            (give the illusion of infinite tape)
            let fallback_empty_tape_cell: TapeCell = TapeCell { val: "#",
prev: this, next: null };
            this.next = fallback_empty_tape_cell;
            return fallback_empty_tape_cell;
        },
        next: {
            return next;
        },
    }
}

fn print_tape(): void {
    // Find start
    let start: TapeCell? = this;
    while (start?.prev != null && start?.prev?.val != "#") {
        start = start?.prev;
    }

    // Print tape
    while (start != null && start?.val != "#") {
        print(start?.val);
        start = start?.next;
    }

    println();
}

struct Transition {
    from_val: str;
    to_val: str;
    direction: int;
    next_state: int;

    // Linked list
    next_elem: Transition?;

    // Add transition to linked list
    fn append(transition: Transition): void {
        match (this.next_elem) {

```

```

        null: { this.next_elem = transition; },
        next: { next.append(transition); }
    }
}

// Find the transition that has a `from_val` that matches the val at the
current tape head
fn find_matching(tape_head_val: str): Transition? {
    if (this.from_val == tape_head_val) {
        return this;
    }

    return this.next_elem?.find_matching(tape_head_val);
}

}

struct State {
    halting: bool;
    transitions: Transition?;

    // Linked list
    next_elem: State?;

    // Add state to linked list
    fn append(state: State): void {
        match (this.next_elem) {
            null: { this.next_elem = state; },
            next: { next.append(state); },
        }
    }

    // Find the matching transition
    fn step(tape_head_val: str): Transition? {
        return this.transitions?.find_matching(tape_head_val);
    }

    // Get the state at a given index
    fn get_at_idx(idx: int): State? {
        if (idx == 0) {
            return this;
        }

        return this.next_elem?.get_at_idx(idx - 1);
    }
}

struct TuringMachine {
    tape_head: TapeCell;
    states: State;
    current_state: State;

    // Returns true if still running (not in halting state)
    fn step(): bool {
        // Find the matching transition
        let transition: Transition? =

```



```

this.current_state.step(this.tape_head.val);
    match (transition) {
        null: { panic("Error - no transition found"); },
        t: {
            // Update the current state
            let next_state: State? = this.states.get_at_idx(t.next_state);
            match (next_state) {
                null: { panic("Error - next state not found"); },
                s: { this.current_state = s; },
            }

            // Update the value on the tape
            this.tape_head.val = t.to_val;

            // If direction is -1, move to left (previous value on tape)
            if (t.direction < 0) {
                this.tape_head = this.tape_head.get_prev_or_extend_empty();
            }
            // If direction is 1, move to right (next value on tape)
            if (t.direction > 0) {
                this.tape_head = this.tape_head.get_next_or_extend_empty();
            }

            // Return true if still running
            return !this.current_state.halting;
        }
    }
}

fn main(): void {
    // Initial tape state `0010111`
    let tape: TapeCell = TapeCell { val: "0", prev: null, next: null };
    tape.append("0");
    tape.append("1");
    tape.append("0");
    tape.append("1");
    tape.append("1");
    tape.append("1");

    // First state transitions:
    // Convert 0 -> 1, move right, stay in same state
    // Convert 1 -> 0, move right, stay in same state
    // Convert # -> #, stay still, move to halting state
    let transitions: Transition = Transition { from_val: "0", to_val: "1",
direction: 1, next_state: 0, next_elem: null };
    transitions.append(Transition { from_val: "1", to_val: "0", direction: 1,
next_state: 0, next_elem: null });
    transitions.append(Transition { from_val: "#", to_val: "#", direction: 0,
next_state: 1, next_elem: null });

    // Create first state with transitions, and second as halting with no
    transitions
    let states: State = State { halting: false, transitions: transitions,
next_elem: null };

```

```

states.append(State { halting: true, transitions: null, next_elem: null });

let tm: TuringMachine = TuringMachine { tape_head: tape, states: states,
current_state: states };

// Loop until halting state
let running: bool = true;
while (running) {
    running = tm.step();
}

tape.print_tape();
}

```

Average transpile time: 3185 microseconds

Initial Tape:

```
0010111
```

Output:

```
1101000
```

B Automated Test Cases

B.1 Example Scanner Tests

```

TEST(ScannerTest, Keyword) {
    std::string source = "if";
    StringScanner scan = StringScanner(source);

    std::optional<Token> next = scan.scan_token();
    EXPECT_EQ(next->t, TokenType::IF);

    next = scan.scan_token();
    EXPECT_EQ(next->t, TokenType::EOF_);
}

TEST(ScannerTest, Identifier) {
    std::string source = "some_identifier1";
    StringScanner scan = StringScanner(source);

    std::optional<Token> next = scan.scan_token();
    EXPECT_EQ(next->t, TokenType::IDENTIFIER);
    EXPECT_EQ(next->lexeme, "some_identifier1");
}

TEST(ScannerTest, MultipleLines) {
    std::string source = "a\nb c\n d\n \"multi\nline\nstring\" e";
    StringScanner scan = StringScanner(source);

    // Expected line numbers

```

```

    auto expected = {1, 2, 2, 3, 4, 6};

    for (auto ex : expected) {
        std::optional<Token> next = scan.scan_token();
        EXPECT_EQ(next->line, ex);
    }
}

TEST(ScannerTest, EqualTokens) {
    std::string source = "<= >= == != < > = !";
    StringScanner scan = StringScanner(source);

    auto expected = {
        TokenType::LESS_EQUAL,
        TokenType::GREATER_EQUAL,
        TokenType::EQUAL_EQUAL,
        TokenType::BANG_EQUAL,
        TokenType::LESS,
        TokenType::GREATER,
        TokenType::EQUAL,
        TokenType::BANG,
    };

    for (auto ex : expected) {
        std::optional<Token> next = scan.scan_token();
        EXPECT_EQ(next->t, ex);
    }
}

TEST(ScannerTest, SkipWhitespace) {
    std::string source = "token1      token2\t\t\n token3      token4 \n token5";
    StringScanner scan = StringScanner(source);

    int token_count = 0;
    std::optional<Token> next = scan.scan_token();
    while (next.has_value() && next->t != TokenType::EOF_) {
        token_count++;
        next = scan.scan_token();
    }

    EXPECT_EQ(token_count, 5);
}

TEST(ScannerTest, Numbers) {
    std::string source = "1 2.345 0.6 789";
    StringScanner scan = StringScanner(source);

    auto expected = {
        std::make_tuple(TokenType::INT_VAL, "1"),
        std::make_tuple(TokenType::FLOAT_VAL, "2.345"),
        std::make_tuple(TokenType::FLOAT_VAL, "0.6"),
        std::make_tuple(TokenType::INT_VAL, "789"),
    };

    for (auto ex : expected) {

```

```

        std::optional<Token> next = scan.scan_token();
        EXPECT_EQ(next->t, std::get<0>(ex));
        EXPECT_EQ(next->lexeme, std::get<1>(ex));
    }
}

TEST(ScannerTest, InvalidNumber) {
    std::string source = "1token";
    StringScanner scan = StringScanner(source);

    EXPECT_DEATH({ scan.scan_token(); }, "Unexpected character in number: '1t'");
}

TEST(ScannerTest, InvalidSymbol) {
    std::string source = "^";
    StringScanner scan = StringScanner(source);

    EXPECT_DEATH({ scan.scan_token(); }, "Unrecognised symbol: '\\^'");
}

TEST(ScannerTest, DoubleTokens) {
    std::string source = "&& || ?? ? & |";
    StringScanner scan = StringScanner(source);

    auto expected = {
        TokenType::AND,
        TokenType::OR,
        TokenType::QUESTION_QUESTION,
        TokenType::QUESTION,
    };

    for (auto ex : expected) {
        std::optional<Token> next = scan.scan_token();
        EXPECT_EQ(next->t, ex);
    }

    EXPECT_DEATH({ scan.scan_token(); }, "Unrecognised symbol: '&'");
    EXPECT_DEATH({ scan.scan_token(); }, "Unrecognised symbol: '|'");
}

TEST(ScannerTest, Types) {
    std::string source = "int str float void bool string boolean";
    StringScanner scan = StringScanner(source);

    auto expected = {
        std::make_tuple(TokenType::TYPE, "int"),
        std::make_tuple(TokenType::TYPE, "str"),
        std::make_tuple(TokenType::TYPE, "float"),
        std::make_tuple(TokenType::TYPE, "void"),
        std::make_tuple(TokenType::TYPE, "bool"),
        std::make_tuple(TokenType::IDENTIFIER, "string"),
        std::make_tuple(TokenType::IDENTIFIER, "boolean"),
    };

    for (auto ex : expected) {

```

```

        std::optional<Token> next = scan.scan_token();
        EXPECT_EQ(next->t, std::get<0>(ex));
        EXPECT_EQ(next->lexeme, std::get<1>(ex));
    }
}

```

B.2 Example Parser Test

```

#define CHECK_AND_CAST(value, type) \
    ({ \
        auto __cast_result = dynamic_cast<type>(value); \
        EXPECT_NE(__cast_result, nullptr); \
        __cast_result; \
    })

// Generator of tokens, wrapping the provided token sequence in a function
struct FunctionWrappedTokenGenerator {
    std::vector<TokenType> token_types;
    int i;

    const std::vector<TokenType> function_tokens = {FN, IDENTIFIER, L_BRACKET,
R_BRACKET, COLON, TYPE, L_CURLY_BRACKET};

    FunctionWrappedTokenGenerator(std::vector<TokenType> token_types)
        : token_types(token_types), i(0) {}

    Token type_to_token(TokenType t) {
        return Token{t, "", -1};
    }

    Token generate_token() {
        if (i < this->function_tokens.size())
            return this->type_to_token(this->function_tokens[i++]);

        if (i < this->function_tokens.size() + this->token_types.size())
            return this->type_to_token(this->token_types[i++ - this->
function_tokens.size()]);

        return this->type_to_token(R_CURLY_BRACKET);
    }
};

TEST(ParserTest, BinaryOperator) {
    std::unique_ptr<MockScanner> scan = std::make_unique<MockScanner>();

    // Generate token sequence in the form `1 + 2.3;`
    FunctionWrappedTokenGenerator gen({INT_VAL, PLUS, FLOAT_VAL, SEMI_COLON});
    EXPECT_CALL(*scan, scan_token).WillRepeatedly(testing::Invoke(&gen,
&FunctionWrappedTokenGenerator::generate_token));

    Parser p = Parser(std::move(scan));
    auto s = p.parse_stmt();

    // Extract expression from function it is wrapped in

```

```

auto fn = CHECK_AND_CAST(s->get(), FunDeclStmt *);
auto expr_stmt = CHECK_AND_CAST(fn->body[0].get(), ExprStmt *);
auto expr = CHECK_AND_CAST(expr_stmt->expr.get(), BinaryExpr *);
EXPECT_EQ(expr->op.t, TokenType::PLUS);

// Check left and right values of the binary operation
auto left = CHECK_AND_CAST(expr->left.get(), LiteralExpr *);
EXPECT_EQ(left->literal.t, TokenType::INT_VAL);

auto right = CHECK_AND_CAST(expr->right.get(), LiteralExpr *);
EXPECT_EQ(right->literal.t, TokenType::FLOAT_VAL);
}

```

B.3 Example Type Checker Tests

```

// Helper function to run the entire front-end of the compiler
std::vector<std::unique_ptr<Stmt>> run_test(std::string &source) {
    auto scan = std::make_unique<StringScanner>(source);
    Parser parser = Parser(std::move(scan));

    std::vector<std::unique_ptr<Stmt>> stmts;
    auto stmt = parser.parse_stmt();
    while (stmt.has_value()) {
        stmts.push_back(std::move(stmt.value()));
        stmt = parser.parse_stmt();
    }

    // Generate type environment
    auto type_env = TypeEnvironment();
    type_env.generate_type_env(stmts);

    // Resolve types
    auto resolver = Resolver(type_env.type_env);
    resolver.resolve(stmts);

    // Check types
    auto type_checker = TypeChecker(type_env.type_env);
    type_checker.check(stmts);

    return std::move(stmts);
}

TEST(TypeCheckerTest, ReturnTypes) {
    // For each example file, `true` means it passes the type checker, `false`
    means it fails
    auto expected = {
        std::make_tuple("returns_success.baz", true),
        std::make_tuple("for_missing_return.baz", false),
        std::make_tuple("if_missing_return.baz", false),
        std::make_tuple("match_missing_return.baz", false),
        std::make_tuple("missing_return.baz", false),
        std::make_tuple("nested_missing_return.baz", false),
        std::make_tuple("returns_no_val_when_void.baz", false),
        std::make_tuple("returns_val_when_void.baz", false),
    };
}

```

```

        std::make_tuple("returns_wrong_type.baz", false),
        std::make_tuple("while_missing_return.baz", false),
    };

    for (auto ex : expected) {
        std::ifstream t(std::string("../test/test_cases/") + std::get<0>(ex));
        std::string source((std::istreambuf_iterator<char>(t)),
                           std::istreambuf_iterator<char>());

        // Ensure if correct types it does not throw, and if incorrect types it
        // does throw
        if (std::get<1>(ex)) {
            EXPECT_NO_THROW({ run_test(source); });
        } else {
            EXPECT_ANY_THROW({ run_test(source); });
        }
    }
}

TEST(TypeCheckerTest, GeneralTypeChecking) {
    // Ensure a complex example with correct types passes
    std::ifstream t(std::string("../test/test_cases/type_checking.baz"));
    std::string source((std::istreambuf_iterator<char>(t)),
                       std::istreambuf_iterator<char>());

    EXPECT_NO_THROW({ run_test(source); });
}

TEST(TypeCheckerTest, Optionals) {
    // For each example file, `true` means it passes the type checker, `false`
    // means it fails
    auto expected = {
        std::make_tuple("optional_type_checking_succeed.baz", true),
        std::make_tuple("optional_type_checking_fail.baz", false),
    };

    for (auto ex : expected) {
        std::ifstream t(std::string("../test/test_cases/") + std::get<0>(ex));
        std::string source((std::istreambuf_iterator<char>(t)),
                           std::istreambuf_iterator<char>());

        // Ensure if correct types it does not throw, and if incorrect types it
        // does throw
        if (std::get<1>(ex)) {
            EXPECT_NO_THROW({ run_test(source); });
        } else {
            EXPECT_ANY_THROW({ run_test(source); });
        }
    }
}

```

Contents of `type_checking.baz` :

```

// ...

```

```

struct SomeObject {
    prop1: int?;
    prop2: bool;

    fn method1(arg1: float): int {
        return 5;
    }

    fn method2(): void {}
}

fn main(): void {
    let obj: SomeObject = SomeObject {
        prop1: null,
        prop2: false,
    };

    let boolean_value: bool = false;

    for (let i: int = 0; i < 100; i = i + 1) {
        if (!boolean_value && obj.method1(12.0) > 1) {
            1 + 3 * 4 / -(obj.prop1 ?? 0);
        }
    }

    let num: int = 1234;
    while (num < 1240) {
        if (num > 0) {
            print("I got here\n");
        } else {
            print("I didn't get there\n");
        }
    }

    for (let i: int = 0; i < 5; i = i + 1) {
        println(i);
    }

    num = num + 1;
}

```

Contents of `optional_type_checking_fail.baz` :

```

fn main(): void {
    let optional: int? = 5;

    // Try assigning a potentially null value to a non-nullable type - should
    fail
    let non_optional: int = optional;
}

```

Other test files can be found in the source code in `test/test_cases/`

C Source Code

All source code is in a [GitHub repository](#), but I have included the core parts below.

C.1 Expressions

`src/ast/expr.h` defines all the possible expressions and the data that they hold. Each expression has a type that is set when resolving and type checking.

```
#pragma once

#include "../scanner/token.h"
#include "../type_checker/type.h"

#include <map>
#include <memory>
#include <optional>
#include <vector>

// Forward declaration - actual implementation will import the visitor
class ExprVisitor;

// Type + whether it is optional
struct TypeInfo {
    std::shared_ptr<Type> type;
    bool optional;

    TypeInfo(std::shared_ptr<Type> type, bool optional);
};

struct Expr {
protected:
    // NOTE: this gets set during resolving/type checking
    std::optional<TypeInfo> type_info;

public:
    virtual void accept(ExprVisitor &visitor) = 0;
    TypeInfo get_type_info();
    void set_type_info(TypeInfo type_info);

    Expr() : type_info(std::nullopt) {}
    virtual ~Expr() = default;
};

//// Types of expressions

struct VarExpr : public Expr {
    Token name;

    VarExpr(Token name);

    void accept(ExprVisitor &visitor) override;
};

struct StructInitExpr : public Expr {
    Token name;
    std::vector<std::tuple<Token, std::unique_ptr<Expr>>> properties;
```

```

    StructInitExpr(Token name, std::vector<std::tuple<Token,
std::unique_ptr<Expr>>> properties);

    void accept(ExprVisitor &visitor) override;
};

struct BinaryExpr : public Expr {
    std::unique_ptr<Expr> left;
    Token op;
    std::unique_ptr<Expr> right;

    BinaryExpr(std::unique_ptr<Expr> left, Token op, std::unique_ptr<Expr>
right);

    void accept(ExprVisitor &visitor) override;
};

struct UnaryExpr : public Expr {
    Token op;
    std::unique_ptr<Expr> right;

    UnaryExpr(Token op, std::unique_ptr<Expr> right);

    void accept(ExprVisitor &visitor) override;
};

struct GetExpr : public Expr {
    std::unique_ptr<Expr> object;
    Token name;
    bool optional;

    GetExpr(std::unique_ptr<Expr> value, Token name, bool optional);

    void accept(ExprVisitor &visitor) override;
};

struct EnumInitExpr : public Expr {
    Token variant;
    std::unique_ptr<VarExpr> enum_namespace;
    std::optional<std::unique_ptr<Expr>> payload;

    EnumInitExpr(Token name, std::unique_ptr<VarExpr> enum_namespace,
std::optional<std::unique_ptr<Expr>> payload);

    void accept(ExprVisitor &visitor) override;
};

struct CallExpr : public Expr {
    std::unique_ptr<Expr> callee;
    std::vector<std::unique_ptr<Expr>> args;

    Token bracket;

    CallExpr(std::unique_ptr<Expr> callee, std::vector<std::unique_ptr<Expr>>

```

```

args, Token bracket);

    void accept(ExprVisitor &visitor) override;
};

struct GroupingExpr : public Expr {
    std::unique_ptr<Expr> expr;

    GroupingExpr(std::unique_ptr<Expr> expr);

    void accept(ExprVisitor &visitor) override;
};

struct LiteralExpr : public Expr {
    Token literal;

    LiteralExpr(Token literal);

    void accept(ExprVisitor &visitor) override;
};

```

C.2 Statements

`src/ast/stmt.h` defines all the possible statements and the data that they hold.

```

#pragma once

#include "../scanner/token.h"
#include "enum_variant.h"
#include "expr.h"
#include "typed_var.h"

#include <optional>
#include <ostream>
#include <variant>
#include <vector>

// Forward declaration - actual implementation will import the visitor
class StmtVisitor;

struct Stmt {
    virtual void accept(StmtVisitor &visitor) = 0;

    virtual ~Stmt() = default;
};

enum FunType {
    FUNCTION,
    METHOD,
};

//// Types of statements

struct FunDeclStmt : public Stmt {

```

```

    Token name;
    Token return_type;
    bool return_type_optional;

    std::vector<TypedVar> params;
    std::vector<std::unique_ptr<Stmt>> body;
    FunType fun_type;

    FunDeclStmt(Token name, std::vector<TypedVar> params, Token return_type, bool
return_type_optional, std::vector<std::unique_ptr<Stmt>> body, FunType fun_type);

    void accept(StmtVisitor &visitor) override;
};

struct EnumMethodDeclStmt : public Stmt {
    Token enum_name;
    std::unique_ptr<FunDeclStmt> fun_definition;

    EnumMethodDeclStmt(std::unique_ptr<FunDeclStmt> fun_definition, Token
enum_name);

    void accept(StmtVisitor &visitor) override;
};

struct StructDeclStmt : public Stmt {
    Token name;
    std::vector<TypedVar> properties;
    std::vector<std::unique_ptr<FunDeclStmt>> methods;

    StructDeclStmt(Token name, std::vector<TypedVar> properties,
std::vector<std::unique_ptr<FunDeclStmt>> methods);

    void accept(StmtVisitor &visitor) override;
};

struct EnumDeclStmt : public Stmt {
    Token name;
    std::vector<EnumVariant> variants;
    std::vector<std::unique_ptr<EnumMethodDeclStmt>> methods;

    EnumDeclStmt(Token name, std::vector<EnumVariant> variants,
std::vector<std::unique_ptr<EnumMethodDeclStmt>> methods);

    void accept(StmtVisitor &visitor) override;
};

struct VariableDeclStmt : public Stmt {
    TypedVar name;
    std::unique_ptr<Expr> initialiser;

    VariableDeclStmt(TypedVar name, std::unique_ptr<Expr> initialiser);

    void accept(StmtVisitor &visitor) override;
};

```

```

struct ExprStmt : public Stmt {
    std::unique_ptr<Expr> expr;

    ExprStmt(std::unique_ptr<Expr> expr);

    void accept(StmtVisitor &visitor) override;
};

struct BlockStmt : public Stmt {
    std::vector<std::unique_ptr<Stmt>> stmts;

    BlockStmt(std::vector<std::unique_ptr<Stmt>> stmts);

    void accept(StmtVisitor &visitor) override;
};

struct IfStmt : public Stmt {
    Token keyword;
    std::unique_ptr<Expr> condition;
    std::vector<std::unique_ptr<Stmt>> true_block;
    std::optional<std::vector<std::unique_ptr<Stmt>>> false_block;

    IfStmt(Token keyword, std::unique_ptr<Expr> condition,
std::vector<std::unique_ptr<Stmt>> true_block,
std::optional<std::vector<std::unique_ptr<Stmt>>> false_block);

    void accept(StmtVisitor &visitor) override;
};

struct EnumPattern {
    Token enum_type;
    Token enum_variant;

    // NOTE: if the enum variant has a payload, this is required
    std::optional<std::unique_ptr<VarExpr>> bound_variable;

    EnumPattern(Token enum_type, Token enum_variant,
std::optional<std::unique_ptr<VarExpr>> bound_variable);
};

struct CatchAllPattern {
    std::unique_ptr<VarExpr> bound_variable;

    CatchAllPattern(std::unique_ptr<VarExpr> bound_variable);
};

struct NullPattern {};

using MatchPattern = std::variant<EnumPattern, NullPattern, CatchAllPattern>;

struct MatchBranch {
    MatchPattern pattern;
    std::vector<std::unique_ptr<Stmt>> body;

    MatchBranch(MatchPattern pattern, std::vector<std::unique_ptr<Stmt>> body);
};

```

```

};

struct MatchStmt : public Stmt {
    std::unique_ptr<Expr> target;
    std::vector<MatchBranch> branches;
    Token keyword;

    MatchStmt(std::unique_ptr<Expr> target, std::vector<MatchBranch> branches,
Token keyword);

    void accept(StmtVisitor &visitor) override;
};

struct WhileStmt : public Stmt {
    std::unique_ptr<Expr> condition;
    std::vector<std::unique_ptr<Stmt>> stmts;

    Token keyword;

    WhileStmt(std::unique_ptr<Expr> condition, std::vector<std::unique_ptr<Stmt>>
stmts, Token keyword);

    void accept(StmtVisitor &visitor) override;
};

struct AssignStmt : public Stmt {
private:
    // NOTE: this gets set when resolving/type checking
    std::optional<TypeInfo> target_type_info;

public:
    Token name;
    std::unique_ptr<Expr> value;
    bool semicolon;

    TypeInfo get_target_type_info();
    void set_target_type_info(TypeInfo type_info);

    AssignStmt(Token name, std::unique_ptr<Expr> value);

    void accept(StmtVisitor &visitor) override;
};

struct ForStmt : public Stmt {
    std::unique_ptr<VariableDeclStmt> var;
    std::unique_ptr<ExprStmt> condition;
    std::unique_ptr<AssignStmt> increment;
    std::vector<std::unique_ptr<Stmt>> stmts;

    ForStmt(std::unique_ptr<VariableDeclStmt> var, std::unique_ptr<ExprStmt>
condition, std::unique_ptr<AssignStmt> increment,
std::vector<std::unique_ptr<Stmt>> stmts);

    void accept(StmtVisitor &visitor) override;
};

```

```

struct PrintStmt : public Stmt {
    std::optional<std::unique_ptr<Expr>> expr;
    bool newline;

    PrintStmt(std::optional<std::unique_ptr<Expr>> expr, bool newline);

    void accept(StmtVisitor &visitor) override;
};

struct PanicStmt : public Stmt {
    std::optional<std::unique_ptr<Expr>> expr;

    PanicStmt(std::optional<std::unique_ptr<Expr>> expr);

    void accept(StmtVisitor &visitor) override;
};

struct ReturnStmt : public Stmt {
    std::optional<std::unique_ptr<Expr>> expr;
    Token keyword;

    ReturnStmt(std::optional<std::unique_ptr<Expr>> expr, Token keyword);

    void accept(StmtVisitor &visitor) override;
};

struct SetStmt : public Stmt {
private:
    // NOTE: this gets set when resolving/type checking
    std::optional<TypeInfo> target_type_info;

public:
    std::unique_ptr<Expr> object;
    Token name;

    std::unique_ptr<Expr> value;

    TypeInfo get_target_type_info();
    void set_target_type_info(TypeInfo type_info);

    SetStmt(std::unique_ptr<Expr> object, Token name, std::unique_ptr<Expr>
value);

    void accept(StmtVisitor &visitor) override;
};

```

C.3 Parser

`src/parser/parser.cpp` handles the parsing of tokens into an abstract syntax tree, build from the expressions and statements above.

```

#include "parser.h"

```

```

#include <iostream>
#include <memory>
#include <optional>
#include <ostream>
#include <vector>

Parser::Parser(std::unique_ptr<Scanner> scanner) : scanner(std::move(scanner)) {
    // Set up first token
    this->advance();
}

std::optional<std::unique_ptr<Stmt>> Parser::parse_stmt() {
    return this->top_level_decl();
}

std::optional<std::unique_ptr<Stmt>> Parser::top_level_decl() {
    if (this->peek().t == TokenType::EOF_)
        return std::nullopt;

    // Can only declare structs, enums, and functions at top level
    if (this->match(TokenType::STRUCT))
        return this->struct_decl();

    if (this->match(TokenType::ENUM))
        return this->enum_decl();

    if (this->match(TokenType::FN))
        return this->function_decl(FunType::FUNCTION);

    this->error(this->peek(), "Unexpected statement at top level.");
    exit(2);
}

// Declarations that can only be done nested inside a function
std::unique_ptr<Stmt> Parser::nested_decl() {
    if (this->match(TokenType::LET))
        return this->variable_decl();

    return this->statement();
}

std::unique_ptr<Stmt> Parser::statement() {
    // Handle each statement based on keyword
    if (this->match(TokenType::IF))
        return this->if_statement();
    if (this->match(TokenType::MATCH))
        return this->match_statement();
    if (this->match(TokenType::FOR))
        return this->for_statement();
    if (this->match(TokenType::WHILE))
        return this->while_statement();
    if (this->match(TokenType::PRINT))
        return this->print_statement();
    if (this->match(TokenType::PANIC))
        return this->panic_statement();
}

```



```

    if (this->match(TokenType::RETURN))
        return this->return_statement();
    if (this->match(TokenType::L_CURLY_BRACKET))
        return std::make_unique<BlockStmt>(std::move(this->block()));

    std::unique_ptr<Expr> expr = this->expression();

    // If expression is followed by `=`, this is an assignment - parse the right
    hand side as an expression
    if (this->match(TokenType::EQUAL)) {
        std::unique_ptr<Stmt> assign_stmt = this->assignment(*expr);
        this->consume(TokenType::SEMI_COLON, "Expected ';' after assignment.");
        return assign_stmt;
    }

    // Expression statement
    this->consume(TokenType::SEMI_COLON, "Expected ';' after expression.");
    return std::make_unique<ExprStmt>(std::move(expr));
}

std::unique_ptr<IfStmt> Parser::if_statement() {
    auto keyword = this->previous();

    this->consume(TokenType::L_BRACKET, "Expected '(' after 'if'.");
    std::unique_ptr<Expr> condition = this->expression();
    this->consume(TokenType::R_BRACKET, "Expected closing ')' after if
condition.");
    this->consume(TokenType::L_CURLY_BRACKET, "Expected '{' before if branch.");

    auto true_block = this->block();

    std::optional<std::vector<std::unique_ptr<Stmt>>> false_block = std::nullopt;
    if (this->match(TokenType::ELSE)) {
        this->consume(TokenType::L_CURLY_BRACKET, "Expected '{' before else
branch.");
        false_block = this->block();
    }

    return std::make_unique<IfStmt>(keyword, std::move(condition),
std::move(true_block), std::move(false_block));
}

std::unique_ptr<MatchStmt> Parser::match_statement() {
    auto keyword = this->previous();
    this->consume(TokenType::L_BRACKET, "Expected '(' after 'match'.");

    std::unique_ptr<Expr> target = this->expression();
    this->consume(TokenType::R_BRACKET, "Expected closing ')' after match
target.");
    this->consume(TokenType::L_CURLY_BRACKET, "Expected '{' before match body.");

    std::vector<MatchBranch> branches;
    do {
        auto pattern = this->match_pattern();
        this->consume(TokenType::COLON, "Expected ':' after match pattern.");
    }

```

```

        this->consume(TokenType::L_CURLY_BRACKET, "Expected block after match
pattern.");
        auto body = this->block();

        branches.push_back(MatchBranch(std::move(pattern), std::move(body)));
    } while (this->match(TokenType::COMMA) && !this-
>check(TokenType::R_CURLY_BRACKET));

    this->consume(TokenType::R_CURLY_BRACKET, "Expected '}' after match
branches");
    return std::make_unique<MatchStmt>(std::move(target), std::move(branches),
keyword);
}

MatchPattern Parser::match_pattern() {
    if (this->match(TokenType::NULL_VAL)) {
        return NullPattern();
    }

    if (!this->match(TokenType::IDENTIFIER)) {
        this->error(this->peek(), "Expected identifier.");
        exit(2);
    }

    auto identifier = this->previous();

    if (!this->match(TokenType::COLON_COLON)) {
        auto payload = std::make_unique<VarExpr>(identifier);
        return CatchAllPattern(std::move(payload));
    }

    if (!this->match(TokenType::IDENTIFIER)) {
        this->error(this->peek(), "Expected variant.");
        exit(2);
    }

    auto enum_variant = this->previous();

    std::optional<std::unique_ptr<VarExpr>> payload = std::nullopt;
    if (this->match(TokenType::L_BRACKET)) {
        if (!this->match(TokenType::IDENTIFIER)) {
            this->error(this->peek(), "Expected variable to bind to.");
            exit(2);
        }

        auto identifier = this->previous();
        payload = std::make_unique<VarExpr>(identifier);
        this->consume(TokenType::R_BRACKET, "Expected ')' after enum payload.");
    }

    return EnumPattern(identifier, enum_variant, std::move(payload));
}

std::unique_ptr<ForStmt> Parser::for_statement() {

```

```

auto keyword = this->previous();

this->consume(TokenType::L_BRACKET, "Expected '(' after 'for'.");
this->consume(TokenType::LET, "Expected variable declaration.");
auto var = this->variable_decl();

auto condition = this->expression();
this->consume(TokenType::SEMI_COLON, "Expected ';' after expression.");
auto condition_stmt = std::make_unique<ExprStmt>(std::move(condition));

auto expr = this->expression();
this->consume(TokenType::EQUAL, "Expected '=' after assignment target.");
auto stmt = this->assignment(*expr);

auto increment = dynamic_cast<AssignStmt*>(stmt.release());
if (!increment) {
    this->error(keyword, "Assign statement not valid.");
}

increment->semicolon = false;

this->consume(TokenType::R_BRACKET, "Expected closing ')' after for
condition.");
this->consume(TokenType::L_CURLY_BRACKET, "Expected '{' before loop body.");
auto body = this->block();
return std::make_unique<ForStmt>(std::move(var), std::move(condition_stmt),
std::unique_ptr<AssignStmt>(increment), std::move(body));
}

std::unique_ptr<WhileStmt> Parser::while_statement() {
    auto keyword = this->previous();
    this->consume(TokenType::L_BRACKET, "Expected '(' after 'while'.");
    std::unique_ptr<Expr> condition = this->expression();
    this->consume(TokenType::R_BRACKET, "Expected closing ')' after while
condition.");
    this->consume(TokenType::L_CURLY_BRACKET, "Expected '{' before loop body.");

    auto body = this->block();
    return std::make_unique<WhileStmt>(std::move(condition), std::move(body),
keyword);
}

std::unique_ptr<PrintStmt> Parser::print_statement() {
    Token print = this->previous();

    this->consume(TokenType::L_BRACKET, "Expected '(' after 'print'.");
    if (this->match(TokenType::R_BRACKET)) {
        this->consume(TokenType::SEMI_COLON, "Expected ';' after print
statement.");
        return std::make_unique<PrintStmt>(
            std::optional<std::unique_ptr<Expr>>{},
            print.lexeme == "println");
    }

    std::unique_ptr<Expr> value = this->expression();

```

```

        this->consume(TokenType::R_BRACKET, "Expected closing ')' after print
value.");
        this->consume(TokenType::SEMI_COLON, "Expected ';' after print statement.");

        return std::make_unique<PrintStmt>(std::move(value), print.lexeme ==
"println");
    }

std::unique_ptr<PanicStmt> Parser::panic_statement() {
    Token print = this->previous();

    this->consume(TokenType::L_BRACKET, "Expected '(' after 'panic'.");
    if (this->match(TokenType::R_BRACKET)) {
        this->consume(TokenType::SEMI_COLON, "Expected ';' after panic
statement.");
        return
std::make_unique<PanicStmt>(std::optional<std::unique_ptr<Expr>>{});
    }

    std::unique_ptr<Expr> value = this->expression();
    this->consume(TokenType::R_BRACKET, "Expected closing ')' after panic
value.");
    this->consume(TokenType::SEMI_COLON, "Expected ';' after panic statement.");

    return std::make_unique<PanicStmt>(std::move(value));
}

std::unique_ptr<ReturnStmt> Parser::return_statement() {
    auto keyword = this->previous();

    if (this->match(TokenType::SEMI_COLON)) {
        return
std::make_unique<ReturnStmt>(std::optional<std::unique_ptr<Expr>>{}, keyword);
    }

    std::unique_ptr<Expr> value = this->expression();
    this->consume(TokenType::SEMI_COLON, "Expected ';' after return statement.");

    return std::make_unique<ReturnStmt>(std::move(value), keyword);
}

std::unique_ptr<Stmt> Parser::assignment(Expr &lhs) {
    Token equals_token = this->previous();
    std::unique_ptr<Expr> value = this->expression();

    if (VarExpr *var = dynamic_cast<VarExpr *>(&lhs)) {
        Token name = var->name;
        return std::make_unique<AssignStmt>(name, std::move(value));
    } else if (GetExpr *get = dynamic_cast<GetExpr *>(&lhs)) {
        return std::make_unique<SetStmt>(std::move(get->object), get->name,
std::move(value));
    }

    this->error(equals_token, "Invalid assignment target.");
    exit(2);
}

```

```

}

std::vector<std::unique_ptr<Stmt>> Parser::block() {
    std::vector<std::unique_ptr<Stmt>> stmts;
    while (!this->check(TokenType::R_CURLY_BRACKET)) {
        stmts.push_back(this->nested_decl());
    }

    this->consume(TokenType::R_CURLY_BRACKET, "Expected '}' after block.");

    return stmts;
}

TypedVar Parser::typed_identifier() {
    Token name = this->consume(TokenType::IDENTIFIER, "Expected identifier.");

    this->consume(TokenType::COLON, "Expected type for identifier.");

    Token type = this->type();
    bool is_optional = this->match(TokenType::QUESTION);

    return TypedVar(name, type, is_optional);
}

Token Parser::type() {
    if (this->match(TokenType::TYPE))
        return this->previous();

    return this->consume(TokenType::IDENTIFIER, "Expected type.");
}

std::unique_ptr<StructDeclStmt> Parser::struct_decl() {
    Token name = this->consume(TokenType::IDENTIFIER, "Expected struct name.");
    this->consume(TokenType::L_CURLY_BRACKET, "Expected '{' before function
body.");

    std::vector<TypedVar> properties;
    std::vector<std::unique_ptr<FunDeclStmt>> methods;

    while (!this->check(TokenType::R_CURLY_BRACKET)) {
        if (this->match(TokenType::FN)) {
            methods.push_back(this->function_decl(FunType::METHOD));
        } else {
            properties.push_back(this->typed_identifier());
            this->consume(TokenType::SEMI_COLON, "Expected ';' after property.");
        }
    };

    this->consume(TokenType::R_CURLY_BRACKET, "Expected '}' after struct body.");

    return std::make_unique<StructDeclStmt>(name, properties,
std::move(methods));
}

std::unique_ptr<EnumDeclStmt> Parser::enum_decl() {

```

```

    Token name = this->consume(TokenType::IDENTIFIER, "Expected enum name.");
    this->consume(TokenType::L_CURLY_BRACKET, "Expected '{' before function
body.");

    std::vector<EnumVariant> variants;
    std::vector<std::unique_ptr<EnumMethodDeclStmt>> methods;

    while (!this->check(TokenType::R_CURLY_BRACKET)) {
        if (this->match(TokenType::FN)) {
            auto fun = this->function_decl(FunType::METHOD);

methods.push_back(std::make_unique<EnumMethodDeclStmt>(std::move(fun), name));
        } else {
            variants.push_back(this->enum_variant_decl());
            this->consume(TokenType::SEMI_COLON, "Expected ';' after property.");
        }
    };

    this->consume(TokenType::R_CURLY_BRACKET, "Expected '}' after struct body.");

    return std::make_unique<EnumDeclStmt>(name, variants, std::move(methods));
}

EnumVariant Parser::enum_variant_decl() {
    Token name = this->consume(TokenType::IDENTIFIER, "Expected name for enum
variant.");

    std::optional<Token> type = std::nullopt;
    bool is_optional = false;
    if (this->match(TokenType::L_BRACKET)) {
        type = this->type();
        is_optional = this->match(TokenType::QUESTION);

        this->consume(TokenType::R_BRACKET, "Expected ')' after variant
payload.");
    }

    return EnumVariant(name, type, is_optional);
}

std::unique_ptr<FunDeclStmt> Parser::function_decl(FunType fun_type) {
    Token name = this->consume(TokenType::IDENTIFIER, "Expected function name.");
    std::vector<TypedVar> params;

    this->consume(TokenType::L_BRACKET, "Expected '(' after function name.");
    if (!this->check(TokenType::R_BRACKET)) {
        // Loop while there are commas
        // If there is a comma followed by a right bracket, it is a trailing
comma
        do {
            params.push_back(this->typed_identifier());
        } while (this->match(TokenType::COMMA) && !this-
>check(TokenType::R_BRACKET));
    }
}

```

```

    this->consume(TokenType::R_BRACKET, "Expected ')' after parameter list.");

    this->consume(TokenType::COLON, "Expected return type.");
    Token return_type = this->type();
    bool return_type_optional = this->match(TokenType::QUESTION);

    // Main function must be type void
    if (fun_type == FunType::FUNCTION && name.lexeme == "main") {
        if (return_type.lexeme != "void") {
            this->error(return_type, "Main function must have return type of
'void'.");
            exit(2);
        }
    }

    this->consume(TokenType::L_CURLY_BRACKET, "Expected '{' before function
body.");
    std::vector<std::unique_ptr<Stmt>> body = this->block();

    return std::make_unique<FunDeclStmt>(name, params, return_type,
return_type_optional, std::move(body), fun_type);
}

std::unique_ptr<VariableDeclStmt> Parser::variable_decl() {
    TypedVar name = this->typed_identifier();
    this->consume(TokenType::EQUAL, "Expected '=' after variable declaration.");
    std::unique_ptr<Expr> expr = this->expression();

    this->consume(TokenType::SEMI_COLON, "Expected ';' after variable
declaration.");

    return std::make_unique<VariableDeclStmt>(name, std::move(expr));
}

std::unique_ptr<Expr> Parser::expression() {
    return this->logical_or();
}

std::unique_ptr<Expr> Parser::logical_or() {
    std::unique_ptr<Expr> expr = this->logical_and();

    while (this->match(TokenType::OR)) {
        Token op = this->previous();
        std::unique_ptr<Expr> rhs = this->logical_and();
        expr = std::make_unique<BinaryExpr>(std::move(expr), op, std::move(rhs));
    }

    return expr;
}

std::unique_ptr<Expr> Parser::logical_and() {
    std::unique_ptr<Expr> expr = this->equality();

    while (this->match(TokenType::AND)) {

```

```

        Token op = this->previous();
        std::unique_ptr<Expr> rhs = this->equality();
        expr = std::make_unique<BinaryExpr>(std::move(expr), op, std::move(rhs));
    }

    return expr;
}

std::unique_ptr<Expr> Parser::equality() {
    std::unique_ptr<Expr> expr = this->comparison();

    if (this->match(TokenType::EQUAL_EQUAL) ||
        this->match(TokenType::BANG_EQUAL)) {
        Token op = this->previous();
        std::unique_ptr<Expr> rhs = this->comparison();
        expr = std::make_unique<BinaryExpr>(std::move(expr), op, std::move(rhs));
    }

    return expr;
}

std::unique_ptr<Expr> Parser::comparison() {
    std::unique_ptr<Expr> expr = this->term();

    if (this->match(TokenType::LESS) || this->match(TokenType::LESS_EQUAL) ||
        this->match(TokenType::GREATER) ||
        this->match(TokenType::GREATER_EQUAL)) {
        Token op = this->previous();
        std::unique_ptr<Expr> rhs = this->term();
        expr = std::make_unique<BinaryExpr>(std::move(expr), op, std::move(rhs));
    }

    return expr;
}

std::unique_ptr<Expr> Parser::term() {
    std::unique_ptr<Expr> expr = this->factor();

    while (this->match(TokenType::PLUS) || this->match(TokenType::MINUS)) {
        Token op = this->previous();
        std::unique_ptr<Expr> rhs = this->factor();
        expr = std::make_unique<BinaryExpr>(std::move(expr), op, std::move(rhs));
    }

    return expr;
}

std::unique_ptr<Expr> Parser::factor() {
    std::unique_ptr<Expr> expr = this->fallback();

    while (this->match(TokenType::STAR) || this->match(TokenType::SLASH)) {
        Token op = this->previous();
        std::unique_ptr<Expr> rhs = this->fallback();
        expr = std::make_unique<BinaryExpr>(std::move(expr), op, std::move(rhs));
    }
}

```



```

    return expr;
}

std::unique_ptr<Expr> Parser::fallback() {
    std::unique_ptr<Expr> expr = this->unary();

    if (this->match(TokenType::QUESTION_QUESTION)) {
        Token op = this->previous();
        std::unique_ptr<Expr> rhs = this->unary();
        expr = std::make_unique<BinaryExpr>(std::move(expr), op, std::move(rhs));
    }

    return expr;
}

std::unique_ptr<Expr> Parser::unary() {
    if (this->match(TokenType::BANG) || this->match(TokenType::MINUS)) {
        Token prev = this->previous();
        return std::make_unique<UnaryExpr>(prev, this->unary());
    }

    return this->call();
}

std::unique_ptr<Expr> Parser::call() {
    std::unique_ptr<Expr> expr = this->primary();

    while (true) {
        if (this->match(TokenType::L_BRACKET)) {
            expr = this->finish_call(std::move(expr));
        } else if (this->match(TokenType::COLON_COLON)) {
            // Enum variant
            Token name = this->consume(TokenType::IDENTIFIER, "Expected variant
name after '.'"");

            auto e = std::unique_ptr<VarExpr>(dynamic_cast<VarExpr
*>(expr.release()));
            if (!e) {
                std::cerr << "[BUG] tried to use variant on non-enum." <<
std::endl;
                exit(3);
            }

            std::optional<std::unique_ptr<Expr>> payload = std::nullopt;

            // If variant has a payload
            if (this->match(TokenType::L_BRACKET)) {
                auto inner = this->expression();
                this->consume(TokenType::R_BRACKET, "Expected ')' after enum
payload.");
                payload = std::move(inner);
            }

            expr = std::make_unique<EnumInitExpr>(

```

```

        name,
        std::move(e),
        std::move(payload));
    } else if (this->match(TokenType::DOT)) {
        // Property access
        Token name = this->consume(TokenType::IDENTIFIER, "Expected property
name after '.'.");
        expr = std::make_unique<GetExpr>(std::move(expr), name, false);
    } else if (this->match(TokenType::QUESTION_DOT)) {
        // Optional property access
        Token name = this->consume(TokenType::IDENTIFIER, "Expected property
name after '.'.");
        expr = std::make_unique<GetExpr>(std::move(expr), name, true);
    } else {
        break;
    }
}

return expr;
}

std::unique_ptr<Expr> Parser::finish_call(std::unique_ptr<Expr> callee) {
    auto bracket = this->previous();
    std::vector<std::unique_ptr<Expr>> args;

    if (!this->check(TokenType::R_BRACKET)) {
        // Loop while there are commas
        // If there is a comma followed by a right bracket, it is a trailing
comma
        do {
            args.push_back(this->expression());
        } while (this->match(TokenType::COMMA) && !this->
check(TokenType::R_BRACKET));
    }

    this->consume(TokenType::R_BRACKET, "Expected ')' after arguments.");

    return std::make_unique<CallExpr>(std::move(callee), std::move(args),
bracket);
}

std::unique_ptr<Expr> Parser::primary() {
    if (this->match(TokenType::IDENTIFIER)) {
        auto identifier = this->previous();
        if (identifier.lexeme != "this" && this->
match(TokenType::L_CURLY_BRACKET))
            return this->finish_struct_init(identifier);

        return std::make_unique<VarExpr>(identifier);
    }

    if (this->match(TokenType::TRUE) || this->match(TokenType::FALSE) ||
this->match(TokenType::NULL_VAL) || this->match(TokenType::INT_VAL) ||
this->match(TokenType::FLOAT_VAL) || this->match(TokenType::STR_VAL)) {
        return std::make_unique<LiteralExpr>(this->previous());
    }
}

```

```

    }

    if (this->match(TokenType::L_BRACKET)) {
        std::unique_ptr<Expr> expr = this->expression();
        this->consume(TokenType::R_BRACKET, "Expected closing ']'");
        return std::make_unique<GroupingExpr>(std::move(expr));
    }

    this->error(this->peek(), "Expected expression.");
    exit(2);
}

std::unique_ptr<Expr> Parser::finish_struct_init(Token name) {
    std::vector<std::tuple<Token, std::unique_ptr<Expr>>> properties;

    // Loop while there are commas
    // If there is a comma followed by a right bracket, it is a trailing comma
    do {
        Token prop_name = this->consume(TokenType::IDENTIFIER, "Expected property
name.");
        this->consume(TokenType::COLON, "Expected property value.");

        std::unique_ptr<Expr> value = this->expression();
        properties.push_back(std::make_tuple(prop_name, std::move(value)));
    } while (this->match(TokenType::COMMA) && !this-
>check(TokenType::R CURLY_BRACKET));

    this->consume(TokenType::R CURLY_BRACKET, "Expected '}' after struct
initialisation.");

    return std::make_unique<StructInitExpr>(name, std::move(properties));
}

// Get next token
Token Parser::advance() {
    this->prev = this->current;
    this->current = scanner->scan_token();

    return this->previous();
}

// Look at current token
Token Parser::peek() {
    return this->current;
}

// Look at previous token
Token Parser::previous() {
    if (!this->prev.has_value()) {
        std::cerr << "[BUG] Expected previous token to exist." << std::endl;
        exit(3);
    }

    return this->prev.value();
}

```

```

// If current token is of the provided type, advance and return true. Else,
return false
bool Parser::match(TokenType t) {
    if (this->check(t)) {
        this->advance();
        return true;
    }

    return false;
}

// Return true if the current token matches the provided token type
bool Parser::check(TokenType t) {
    Token curr = this->peek();
    return curr.t == t;
}

// Expect the current token to be of type `t`, otherwise error and output the
provided message
Token Parser::consume(TokenType t, std::string error_message) {
    if (this->check(t))
        return this->advance();

    this->error(this->peek(), error_message);
    exit(2);
}

// Error reporting with line number
void Parser::error(Token error_token, std::string message) {
    std::string where = "end";
    if (error_token.t != TokenType::EOF_)
        where = "'" + error_token.lexeme + "'";

    std::cerr << "[line " << error_token.line << "] Syntax error at " << where <<
    ": " << message << std::endl;
}

```

C.4 Type Checking

src/type_checker/type_checker.cpp contains all the code for ensuring types are correct, and also stores their types on the AST.

```

#include "type_checker.h"
#include "type.h"

#include <algorithm>
#include <iostream>
#include <memory>

bool can_coerce_to(std::shared_ptr<Type> from, bool from_optional,
std::shared_ptr<Type> to, bool to_optional) {
    // If going from an optional to non-optional, it cannot coerce
    if (from_optional && !to_optional) {

```

```

        return false;
    }

    // Allow `null` to coerce to any optional type
    if (to_optional && from->type_class == TypeClass::NULL_)
        return true;

    // Check if the type itself can coerce
    return from->can_coerce_to(to);
}

TypeChecker::TypeChecker(std::map<std::string, std::shared_ptr<Type>> type_env) :
    type_env(type_env), result(TypeInfo(nullptr, false)) {}

bool TypeChecker::is_numeric(Type *t) {
    return t->can_coerce_to(this->type_env["int"]) || t->can_coerce_to(this->type_env["float"]);
}

// Check all statements have valid types
void TypeChecker::check(std::vector<std::unique_ptr<Stmt>> &stmts) {
    for (auto &stmt : stmts) {
        stmt->accept(*this);
    }
}

// Error message with line number
void TypeChecker::error(Token error_token, std::string message) {
    std::string where = "end";
    if (error_token.t != TokenType::EOF_)
        where = "'" + error_token.lexeme + "'";

    std::cerr << "[line " << error_token.line << "] Type error at " << where <<
    ": " << message << std::endl;

    throw TypeCheckerError::OP_ON_INCOMPATIBLE_TYPES;
}

//// Expressions

void TypeChecker::visit_var_expr(VarExpr *expr) {
    this->result = expr->get_type_info();
    expr->set_type_info(this->result);
    this->always_returns = false;
}

void TypeChecker::visit_struct_init_expr(StructInitExpr *expr) {
    if (auto t = std::dynamic_pointer_cast<StructType>(expr->get_type_info().type)) {
        if (expr->properties.size() != t->props.size())
            this->error(expr->name, "Expected " + std::to_string(t->props.size())
            + " arguments, received " + std::to_string(expr->properties.size()) + ".");

        // Loop through in order of declared type props
        for (auto &prop : t->props) {

```

```

        auto name = prop.name.lexeme;

        // Find corresponding property
        auto p = std::find_if(expr->properties.begin(), expr-
>properties.end(), [name](const auto &p) {
            return std::get<0>(p).lexeme == name;
        });

        if (p == expr->properties.end())
            this->error(expr->name, "Missing property " + name + " from
constructor.");

        auto from = std::get<1>(*p)->get_type_info();
        auto to = this->type_env[prop.type.lexeme];
        if (!can_coerce_to(from.type, from.optional, to, prop.is_optional)) {
            this->error(std::get<0>(*p), "Cannot assign a type '" +
from.type->to_string() + (from.optional && from.type->type_class !=
TypeClass::NULL_ ? "?" : "") + "' to variable of type '" + to->to_string() +
(prop.is_optional ? "?" : "") + "'.");
        }
    }
} else {
    this->error(expr->name, "Cannot initialise non-struct.");
}

this->result = expr->get_type_info();
expr->set_type_info(this->result);
this->always_returns = false;
}

void TypeChecker::visit_binary_expr(BinaryExpr *expr) {
    switch (expr->op.t) {
        case TokenType::PLUS:
        case TokenType::MINUS:
        case TokenType::STAR:
        case TokenType::SLASH: {
            // Check left and right are the same type
            expr->left->accept(*this);
            auto left_t = this->result;

            // If not numeric or string, we can't operate
            if (left_t.optional || !(is_numeric(left_t.type.get()) ||
left_t.type.get()->type_class == TypeClass::STR))
                this->error(expr->op, "Operator can only be used on numeric or
string types.");

            expr->right->accept(*this);
            auto right_t = this->result;

            if (!can_coerce_to(right_t.type, right_t.optional, left_t.type,
left_t.optional))
                this->error(expr->op, "Operands must be the same type, or
coercible to the same type.");

            this->result = left_t;
        }
    }
}

```

```

    expr->set_type_info(this->result);
    this->always_returns = false;
    return;
}
case TokenType::LESS:
case TokenType::LESS_EQUAL:
case TokenType::GREATER:
case TokenType::GREATER_EQUAL: {
    // Check left and right are the same type, and numeric
    expr->left->accept(*this);
    auto left_t = this->result;

    // If not numeric, we can't compare
    if (left_t.optional || !is_numeric(left_t.type.get()))
        this->error(expr->op, "Operator can only be used on numeric
types.");

    expr->right->accept(*this);
    auto right_t = this->result;

    if (!can_coerce_to(right_t.type, right_t.optional, left_t.type,
left_t.optional))
        this->error(expr->op, "Operands must be the same type, or
coercible to the same type.");

    this->result = TypeInfo(this->type_env["bool"], false);
    expr->set_type_info(this->result);
    this->always_returns = false;
    return;
}
case TokenType::QUESTION_QUESTION: {
    // Check left and right are the same type
    expr->left->accept(*this);
    auto left_t = this->result;

    expr->right->accept(*this);
    auto right_t = this->result;

    if (!left_t.optional)
        this->error(expr->op, "Cannot use '?\?'" operator on a non-
nullable type.");

    // Has to coerce to non-nullable version of left type
    if (!can_coerce_to(right_t.type, right_t.optional, left_t.type,
false))
        this->error(expr->op, "Operands must be the same type, or
coercible to the same type.");

    // Always go to non-optional form of left type
    this->result = left_t;
    this->result.optional = false;
    expr->set_type_info(this->result);

    this->always_returns = false;
    return;
}

```

```

    }
    case TokenType::EQUAL_EQUAL:
    case TokenType::BANG_EQUAL: {
        expr->left->accept(*this);
        auto left_t = this->result;

        expr->right->accept(*this);
        auto right_t = this->result;

        // Left can coerce to right or right to left
        if (!(can_coerce_to(right_t.type, right_t.optional, left_t.type,
left_t.optional) || can_coerce_to(left_t.type, left_t.optional, right_t.type,
right_t.optional)))
            this->error(expr->op, "Operands must be the same type, or
coercible to the same type.");

        this->result = TypeInfo(this->type_env["bool"], false);
        expr->set_type_info(this->result);
        this->always_returns = false;
        return;
    }
    case TokenType::AND:
    case TokenType::OR: {
        expr->left->accept(*this);
        auto left_t = this->result;

        auto from = this->result;
        auto to = this->type_env["bool"];
        if (!can_coerce_to(from.type, from.optional, to, false))
            this->error(expr->op, "Operator can only be used on boolean
types.");

        expr->right->accept(*this);
        auto right_t = this->result;

        if (left_t.type != right_t.type)
            this->error(expr->op, "Operands must be the same type, or
coercible to the same type.");

        this->result = TypeInfo(this->type_env["bool"], false);
        expr->set_type_info(this->result);
        this->always_returns = false;
        return;
    }
    default:
        std::cerr << "[BUG] Binary operator type '" <<
get_token_type_str(expr->op.t) << "' not handled." << std::endl;
        exit(3);
    }
}
}

void TypeChecker::visit_unary_expr(UnaryExpr *expr) {
    expr->right->accept(*this);

    switch (expr->op.t) {

```



```

        case TokenType::BANG: {
            // If not boolean, we can't invert
            auto from = this->result;
            auto to = this->type_env["bool"];
            if (!can_coerce_to(from.type, from.optional, to, false))
                this->error(expr->op, "Operator can only be used on a boolean
type.");

            break;
        }
        case TokenType::MINUS:
            // If not numeric, we can't invert
            if (this->result.optional || !is_numeric(this->result.type.get()))
                this->error(expr->op, "Operator can only be used on numeric
types.");

            break;
        default:
            std::cerr << "[BUG] Unary operator type '" <<
get_token_type_str(expr->op.t) << "' not handled." << std::endl;
            exit(3);
    }

    expr->set_type_info(this->result);
    this->always_returns = false;
}

void TypeChecker::visit_get_expr(GetExpr *expr) {
    expr->object->accept(*this);
    if (expr->optional && !this->result.optional) {
        this->error(expr->name, "Cannot use optional chaining on non-optional
type.");
    }

    if (!expr->optional && this->result.optional) {
        this->error(expr->name, "Must use optional chaining for optional type.");
    }

    this->result = expr->get_type_info();
    expr->set_type_info(this->result);
    this->always_returns = false;
}

void TypeChecker::visit_enum_init_expr(EnumInitExpr *expr) {
    // Check payload if there is one
    if (expr->payload.has_value()) {
        expr->payload.value()->accept(*this);

        // Cast to enum type (should always be enum type)
        if (auto t = std::dynamic_pointer_cast<EnumType>(expr-
>get_type_info().type)) {
            // Check type of payload for this variant
            auto payload_type_info = t->get_variant_payload_type(expr-
>variant.lexeme);
            auto payload_type = this->type_env[payload_type_info->type.lexeme];

```

```

        if (!can_coerce_to(this->result.type, this->result.optional,
payload_type, payload_type_info->optional)) {
            this->error(expr->variant, "Payload of enum cannot be coerced to
a valid type.");
        }

        if (this->result.optional && !payload_type_info->optional) {
            this->error(expr->variant, "Expected non-optional type. Received
optional type.");
        }
    } else {
        std::cerr << "[BUG] Enum doesn't have enum type" << std::endl;
        exit(3);
    }
}

this->result = expr->get_type_info();
expr->set_type_info(this->result);
this->always_returns = false;
}

void TypeChecker::visit_call_expr(CallExpr *expr) {
    expr->callee->accept(*this);

    // Check we are calling a function
    auto t = std::dynamic_pointer_cast<FunctionType>(expr->callee-
>get_type_info().type);
    if (!t) {
        this->error(expr->bracket, "Cannot call non-function.");
    }

    // Check the number of arguments equals the number of parameters
    if (expr->args.size() != t->params.size()) {
        this->error(expr->bracket, "Expected " + std::to_string(t->params.size())
+ " arguments, received " + std::to_string(expr->args.size()) + ".");
    }

    // Check that all parameters have the correct type
    for (int i = 0; i < expr->args.size(); i++) {
        expr->args[i]->accept(*this);
        if (!this->result.type->can_coerce_to(this->type_env[t-
>params[i].type.lexeme])) {
            this->error(expr->bracket, "Invalid type passed to function.");
        }

        if (this->result.optional && !t->params[i].is_optional) {
            this->error(expr->bracket, "Expected non-optional type. Received
optional type.");
        }
    }

    this->result = expr->get_type_info();
    expr->set_type_info(this->result);

    // If we are optional chaining, carry forwards optional type

```

```

    this->result.optional |= expr->callee->get_type_info().optional;
    this->always_returns = false;
}

void TypeChecker::visit_grouping_expr(GroupingExpr *expr) {
    expr->expr->accept(*this);
    expr->set_type_info(this->result);
    this->always_returns = false;
}

void TypeChecker::visit_literal_expr(LiteralExpr *expr) {
    this->result = expr->get_type_info();
    expr->set_type_info(this->result);
    this->always_returns = false;
}

//// Statements

void TypeChecker::visit_fun_decl_stmt(FunDeclStmt *fun) {
    auto prev_fn_ret_type = this->surrounding_fn_return_type;
    this->surrounding_fn_return_type = OptionalTypeInfo{
        fun->return_type,
        fun->return_type_optional,
    };

    // Ensure at least one line of the body returns
    bool returns = false;
    for (auto &s : fun->body) {
        s->accept(*this);
        returns = returns || this->always_returns;
    }

    this->surrounding_fn_return_type = prev_fn_ret_type;

    // If not void, it must return a value from every path
    if (fun->return_type.lexeme != "void" && !returns) {
        this->error(fun->name, "Not all code paths return.");
    }

    this->always_returns = false;
}

void TypeChecker::visit_enum_method_decl_stmt(EnumMethodDeclStmt *fun) {
    fun->fun_definition->accept(*this);

    this->always_returns = false;
}

void TypeChecker::visit_struct_decl_stmt(StructDeclStmt *stmt) {
    for (auto &method : stmt->methods) {
        method->accept(*this);
    }

    this->always_returns = false;
}

```

```

void TypeChecker::visit_enum_decl_stmt(EnumDeclStmt *stmt) {
    for (auto &method : stmt->methods) {
        method->accept(*this);
    }

    this->always_returns = false;
}

void TypeChecker::visit_variable_decl_stmt(VariableDeclStmt *stmt) {
    stmt->initialiser->accept(*this);

    auto from = this->result;
    auto to = this->type_env[stmt->name.type.lexeme];
    if (!can_coerce_to(from.type, from.optional, to, stmt->name.is_optional)) {
        this->error(stmt->name.name, "Cannot assign a type '" + from.type-
>to_string() + (from.optional && from.type->type_class != TypeClass::NULL_ ?
"?" : "") + "' to variable of type '" + to->to_string() + (stmt-
>name.is_optional ? "?" : "") + "'");
    }

    this->always_returns = false;
}

void TypeChecker::visit_expr_stmt(ExprStmt *stmt) {
    stmt->expr->accept(*this);
    this->always_returns = false;
}

void TypeChecker::visit_block_stmt(BlockStmt *stmt) {
    bool returns = false;

    for (auto &s : stmt->stmts) {
        s->accept(*this);

        // If any statement always returns, the block always returns
        returns = returns || this->always_returns;
    }

    this->always_returns = returns;
}

void TypeChecker::visit_if_stmt(IfStmt *stmt) {
    stmt->condition->accept(*this);

    auto from = this->result;
    auto to = this->type_env["bool"];
    if (!can_coerce_to(from.type, from.optional, to, false)) {
        this->error(stmt->keyword, "If condition must be a boolean value.");
    }

    bool true_block_returns = false;
    for (auto &line : stmt->true_block) {
        line->accept(*this);
        true_block_returns = true_block_returns || this->always_returns;
    }
}

```

```

    }

    bool false_block_returns = false;
    if (stmt->>false_block.has_value()) {
        for (auto &line : stmt->>false_block.value()) {
            line->accept(*this);
            false_block_returns = false_block_returns || this->always_returns;
        }
    }

    this->always_returns = true_block_returns && false_block_returns;
}

void TypeChecker::visit_match_stmt(MatchStmt *stmt) {
    bool all_branches_return = true;

    auto t = std::dynamic_pointer_cast<EnumType>(stmt->target->get_type_info().type);
    if (!(t || stmt->target->get_type_info().optional)) {
        this->error(stmt->keyword, "Can only match on enum or optional.");
    }

    if (t) {
        // Enum
        if (stmt->branches.size() != (t->variants.size() + (stmt->target->get_type_info().optional ? 1 : 0))) {
            this->error(stmt->keyword, "Not all variants covered in pattern matching.");
        }

        bool has_null_branch = false;
        for (auto &branch : stmt->branches) {
            if (std::holds_alternative<EnumPattern>(branch.pattern)) {
                // Check that match pattern is valid and matches the type of the
                target

                auto &enum_pattern = std::get<EnumPattern>(branch.pattern);
                if (enum_pattern.enum_type.lexeme != t->name.lexeme) {
                    this->error(enum_pattern.enum_type, "Match pattern must be
for the same enum type.");
                }

                auto variant_name = enum_pattern.enum_variant.lexeme;
                auto v = std::find_if(t->variants.begin(), t->variants.end(),
[variant_name](const auto &v) {
                    return v.name.lexeme == variant_name;
                });

                if (v == t->variants.end()) {
                    this->error(enum_pattern.enum_type, "Could not find variant
on enum type.");
                }

                if (v->payload_type.has_value()) {
                    if (!enum_pattern.bound_variable.has_value()) {
                        this->error(enum_pattern.enum_variant, "Enum variant

```

```

expects payload.");
    }
    } else {
        if (enum_pattern.bound_variable.has_value()) {
            this->error(enum_pattern.enum_type, "No payload available
for this enum variant.");
        }
    }
} else if (std::holds_alternative<NullPattern>(branch.pattern)) {
    has_null_branch = true;
} else {
    this->error(stmt->keyword, "Can only match enum variant or null
on enum.");
}

bool returns = false;
for (auto &line : branch.body) {
    line->accept(*this);
    returns = returns || this->always_returns;
}

if (!returns)
    all_branches_return = false;
}

if (stmt->target->get_type_info().optional && !has_null_branch) {
    this->error(stmt->keyword, "Null variant not provided.");
}

this->always_returns = all_branches_return;
} else {
    // Either null or bound variable
    if (stmt->branches.size() != 2) {
        this->error(stmt->keyword, "Not all variants covered in pattern
matching.");
    }

    bool has_null_branch = false;
    bool has_catch_all_branch = false;
    for (auto &branch : stmt->branches) {
        if (std::holds_alternative<CatchAllPattern>(branch.pattern)) {
            has_catch_all_branch = true;
        } else if (std::holds_alternative<NullPattern>(branch.pattern)) {
            has_null_branch = true;
        } else {
            this->error(stmt->keyword, "Can only match existing or null on
enum.");
        }
    }

    bool returns = false;
    for (auto &line : branch.body) {
        line->accept(*this);
        returns = returns || this->always_returns;
    }
}

```

```

        if (!returns)
            all_branches_return = false;
    }

    if (!has_null_branch || !has_catch_all_branch) {
        this->error(stmt->keyword, "Not all variants covered in pattern
matching.");
    }

    this->always_returns = all_branches_return;
}

void TypeChecker::visit_while_stmt(WhileStmt *stmt) {
    stmt->condition->accept(*this);

    auto from = this->result;
    auto to = this->type_env["bool"];
    if (!can_coerce_to(from.type, from.optional, to, false)) {
        this->error(stmt->keyword, "While condition must be a boolean value.");
    }

    bool returns = false;
    for (auto &s : stmt->stmts) {
        s->accept(*this);
        returns = returns || this->always_returns;
    }

    this->always_returns = returns;
}

void TypeChecker::visit_for_stmt(ForStmt *stmt) {
    stmt->var->accept(*this);
    stmt->condition->accept(*this);
    stmt->increment->accept(*this);

    bool returns = false;
    for (auto &s : stmt->stmts) {
        s->accept(*this);
        returns = returns || this->always_returns;
    }

    this->always_returns = returns;
}

void TypeChecker::visit_print_stmt(PrintStmt *stmt) {
    if (stmt->expr.has_value())
        stmt->expr.value()->accept(*this);

    this->always_returns = false;
}

void TypeChecker::visit_panic_stmt(PanicStmt *stmt) {
    if (stmt->expr.has_value())
        stmt->expr.value()->accept(*this);
}

```

```

    this->always_returns = true;
}

void TypeChecker::visit_return_stmt(ReturnStmt *stmt) {
    if (!this->surrounding_fn_return_type.has_value()) {
        this->error(stmt->keyword, "Cannot return from outside of a function.");
    }

    if (stmt->expr.has_value()) {
        stmt->expr.value()->accept(*this);

        if (!can_coerce_to(this->result.type, this->result.optional, this->
type_env[this->surrounding_fn_return_type->type.lexeme], this->
surrounding_fn_return_type->optional)) {
            this->error(stmt->keyword, "Must return a type that equals or can
coerce to the return type of the function.");
        }
    } else {
        if (this->surrounding_fn_return_type->type.lexeme != "void") {
            this->error(stmt->keyword, "Must return a value from a non-void
function.");
        }
    }

    this->always_returns = true;
}

void TypeChecker::visit_assign_stmt(AssignStmt *stmt) {
    stmt->value->accept(*this);

    auto from = this->result;
    auto to = stmt->get_target_type_info();
    if (!can_coerce_to(from.type, from.optional, to.type, to.optional)) {
        this->error(stmt->name, "Cannot assign a different type to this
variable.");
    }

    this->always_returns = false;
}

void TypeChecker::visit_set_stmt(SetStmt *stmt) {
    stmt->value->accept(*this);

    auto from = this->result;
    auto to = stmt->get_target_type_info();
    if (!can_coerce_to(from.type, from.optional, to.type, to.optional)) {
        this->error(stmt->name, "Cannot assign a different type to this
variable.");
    }

    this->always_returns = false;
}

```